

EDUDEMOS



Demostrador 3×1:

Energía Solar, eólica y captación de agua
en un solo proyecto educativo

Financiado por la Unión Europea. Las opiniones y puntos de vista expresados solo comprometen a su(s) autor(es) y no reflejan necesariamente los de la Unión Europea o los de la Agencia Ejecutiva Europea de Educación y Cultura (EACEA). Ni la Unión Europea ni la EACEA pueden ser considerados responsables de ellos.



EduDemoS 3in1 © 2025 by Gerda Stetter Stiftung – Technik macht Spaß!, Sergio Alonso Fundation, FINNOVA REGIO, GBS St. Gallen e IES El Rincón is licensed under

Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International. To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/>

1. ENSAMBLAJE DEL DEMOSTRADOR 3x1

El montaje del Demostrador 3x1 se realizará de forma modular, abordando cada sección por separado en los apartados correspondientes de esta guía. Este enfoque busca simplificar el proceso y facilitar su comprensión, permitiendo que el montaje sea claro y accesible para quienes lo lleven a cabo. Cada paso está diseñado con el objetivo de garantizar un resultado funcional y efectivo, incluso para quienes no cuenten con experiencia previa en ensamblaje de demostradores.

Uno de los objetivos principales de este proyecto es fomentar la autonomía de los centros educativos, permitiéndoles crear sus propios demostradores si cuentan con impresoras 3D en sus instalaciones. Por esta razón, hemos puesto a disposición los archivos de impresión necesarios para la fabricación de los componentes, los cuales pueden descargarse a través del siguiente enlace: [Archivos de impresión 3D](#). Con esta herramienta, buscamos empoderar a las instituciones educativas para que integren estos demostradores en sus actividades de forma independiente y eficiente.

1.1. Listado de materiales

- 1 x **ESP32 DEVKIT V1** de 30 pines
- 2 x **Mini breadboard** de 170 puntos
- 1 x **Módulo DHT11**, de ancho máximo 14 mm y puede o no tener led incluido
- 10/12* x **Cables macho-macho** 10 cm
- 5 x **Cables macho-macho** 20 cm
- 3 x **Cables macho-hembra** 10 cm
- 10 x **Cables macho-hembra** 20 cm
- 5 x **Resistencias** de 1k ohmio
- 1 x **Condensador** de 100nF / 1uF
- 3 x **Leds** de colores (Rojo, Verde y Azul preferiblemente)
- 1 x **Placa solar** dimensiones máximas: 45 mm x 60 mm
- 1 x **Sensor nivel de agua**, 6 cm x 2 cm
- 1 x **Motor DC** de corriente continua de 5V, dimensiones máximas: 28 mm diámetro y 45 mm largo
- 1 x **Servomotor SG-90**

- 1 x **Detector de tensión**
- 2 x **Fotoresistencias** independientes GL5528
- 1 x **Rodamiento 6203** puede ser del tipo zz o 2rs: 40 mm diámetro exterior, 17 mm diámetro interior y 12 mm de alto
- 3 x **Espiga de madera**: 8 mm de diámetro y 40mm de largo
- 1 x **Engranaje motor**: 2 mm diámetro interior, 10 dientes y 5 mm alto
- 1 x **Cable de datos USB a microUSB**
- 1 x **Cinta adhesiva de aluminio**, 5 cm de ancho
- 1 x **Cinta adhesiva de doble cara**, 3 cm de ancho

* Si el Motor DC no viene con los cables soldados, se necesitarán 12 cables macho-macho de 10 cm.

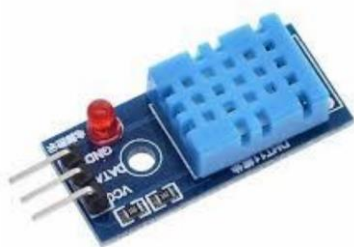
Herramientas que podrías necesitar:

- **Soldador**: si el motor DC o el panel solar no vienen con cables.
- **Destornillador plano de 2 mm**: para el sensor de voltaje.

1. 2. Montaje del Demostrador de Agua + Solar

1. 2. 1. Montaje electrónico (Parte de Agua)

Comenzaremos realizando el montaje del captador de agua. Para realizar este montaje completo necesitarás el siguiente material:



1 x Sensor DHT-11



3 x Resistencias de 1 kΩ

Encargado de medir la temperatura y la humedad ambientales.

Usadas para limitar la corriente que fluye por los diodos LED.



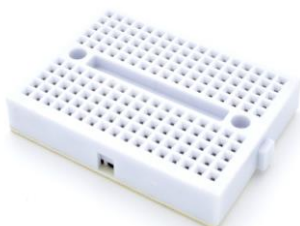
3 x LEDs de Colores (Rojo, Verde y Azul preferiblemente)

Empleados como indicadores visuales del rango de la temperatura medida.



1 x Cable dupont (Macho-Macho) 10cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



2 x Mini Breadboard

Componente necesario para conectar la placa microcontroladora con los diferentes componentes



6 x Cable dupont (Macho-Hembra) 20cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



1 x Sensor nivel de agua

Empleado para medir el nivel de agua en el tanque.

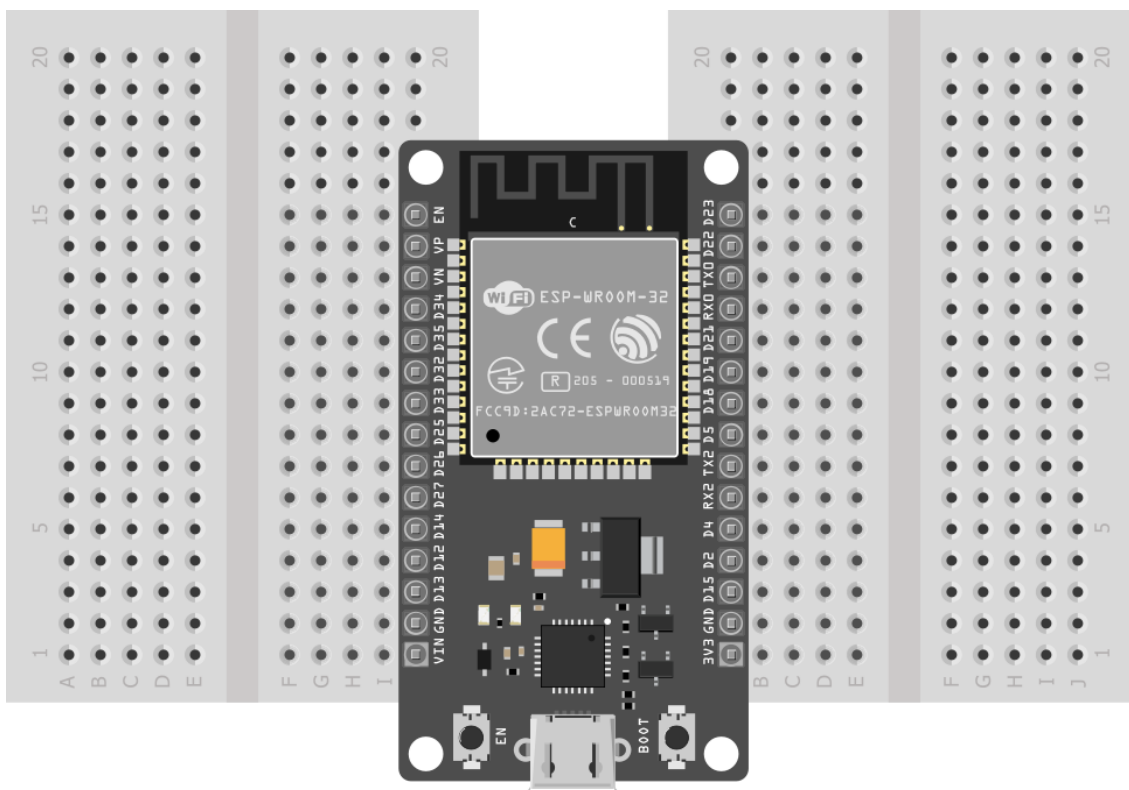


1 x ESP32 DEVKIT V1 (30 pins)

Encargado de ejecutar la programación que controla el funcionamiento de la electrónica.

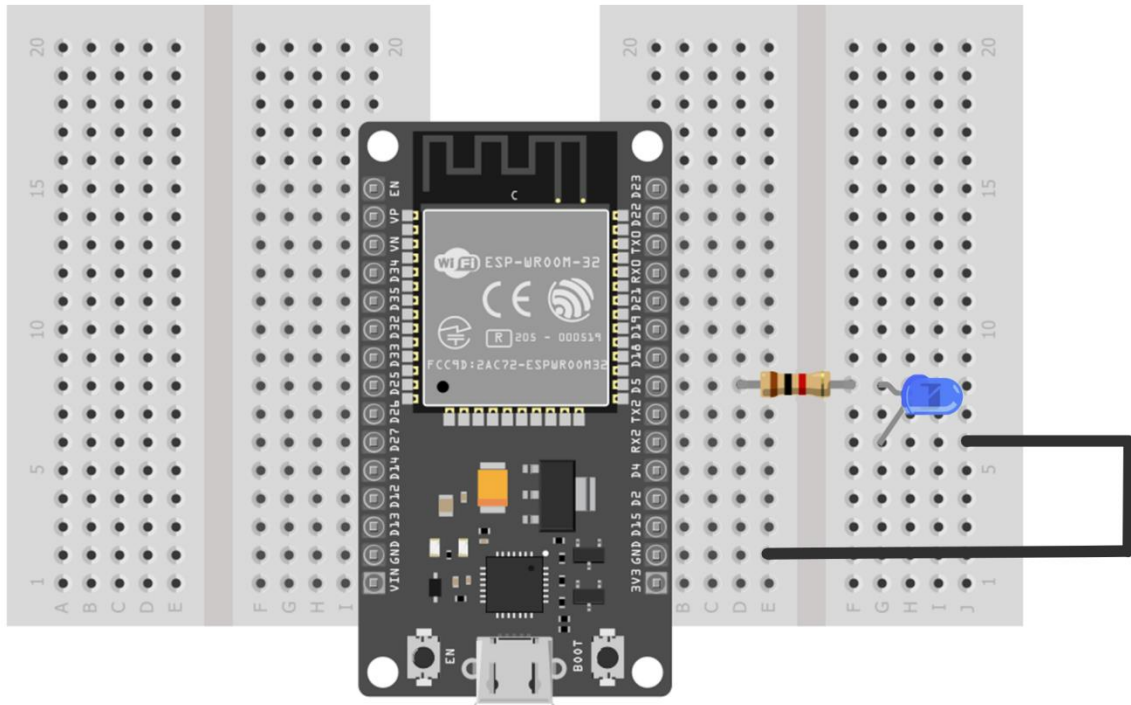
El objetivo del montaje será obtener el valor de la temperatura y la humedad ambientales y, en función del valor de la temperatura, encender un LED u otro como indicadores de peligro por alta o baja temperatura. De esta manera si la temperatura está por debajo de los 20°C se enciende el LED Azul, si se está por encima de los 25°C se enciende el LED Rojo y si está entre ambos valores (entre 20 y 25°C) se enciende el LED Verde.

El primer paso es colocar la placa con ESP32 sobre dos protoboard pequeñas para tener espacio suficiente para luego conectar los componentes que vamos a necesitar. A continuación se puede ver el resultado:



Una vez tengamos el microcontrolador colocado en las dos protoboards, empezaremos a conectar los LEDs de colores con sus resistencias limitadoras. Vamos a empezar colocando el LED Azul y una de las resistencias de 1 k Ω en serie. Conectaremos el positivo del LED en el **pin D5**, y utilizaremos cualquier pin GND para conectar el negativo del circuito. A continuación se puede ver el montaje indicado.

Nota: El diodo LED nos indica la polaridad de la conexión a simple vista con el tamaño de sus patas de conexión. La más larga coincide con el ánodo, o positivo, y la más corta se trata del cátodo, o negativo. En el diagrama, la pata larga es la que va conectada en serie con la resistencia.



Para comprobar el correcto funcionamiento del montaje, podemos cargar el siguiente código en el ESP32:

```
#define ledPin1Cold 5 // Definimos el pin donde está conectado el LED Azul

// Bloque de inicialización
void setup() {

    // Configuramos el modo de funcionamiento del pin del LED como salida
    pinMode(ledPin1Cold, OUTPUT);

}

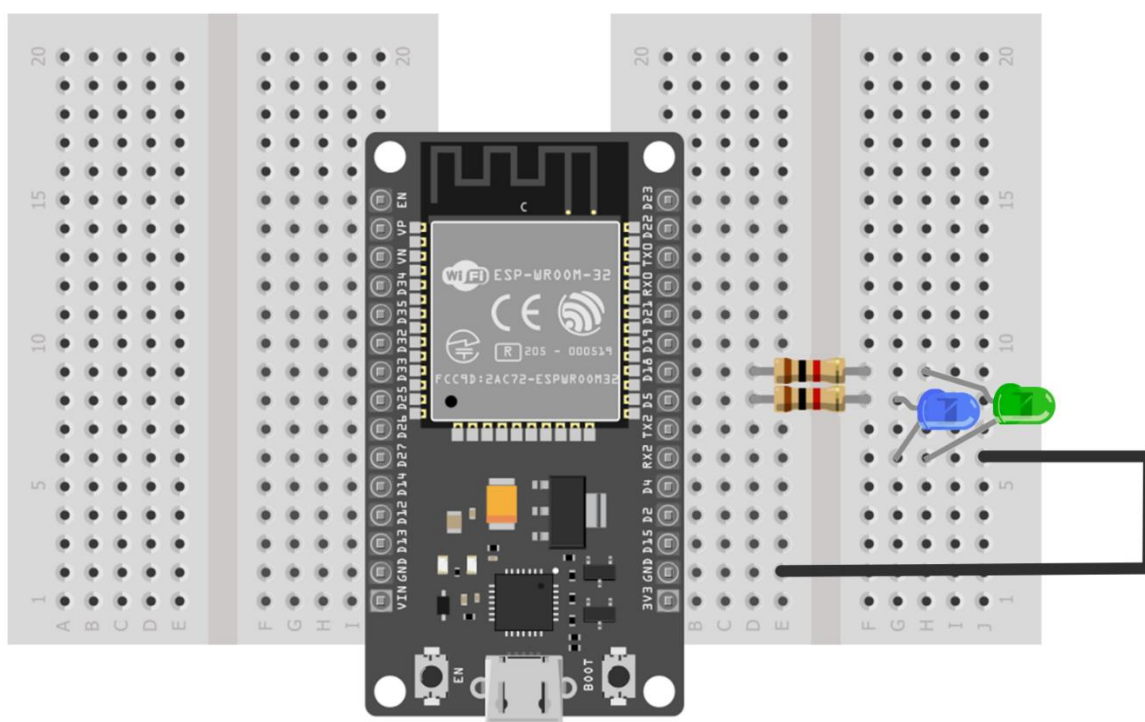
// Bucle principal del código
void loop() {

    digitalWrite(ledPin1Cold, LOW); // Apagamos el LED
    delay(1000); // Esperamos 1 segundo = 1000 milisegundos
    digitalWrite(ledPin1Cold, HIGH); // Encendemos el LED
    delay(1000); // Esperamos 1 segundo
```


}

Este pequeño programa hace parpadear el LED Azul, cambiando su estado entre encendido y apagado cada segundo. Si no funciona correctamente, comprueba que el LED se encuentre bien orientado (recuerda que el terminal más largo es el positivo, mientras el corto es el negativo) y conectado adecuadamente con la resistencia limitadora.

El siguiente paso del montaje será añadir un segundo LED. Esta vez colocaremos y conectaremos el LED de color Verde en el **pin D18** con su resistencia. Puedes ver el esquema de montaje indicado en la siguiente imagen.



Vamos a comprobar de nuevo que el montaje es correcto. Vamos a utilizar ahora el código que encontrarás a continuación para verificar que el nuevo LED también está adecuadamente colocado.

```
#define ledPin1Cold 5 // Definimos el pin donde está conectado el LED Azul
#define ledPin2Good 18 // Definimos el pin donde está conectado el LED Verde

// Bloque de inicialización
void setup() {

    // Configuramos el modo de funcionamiento de los pines de los LED como salida
    pinMode(ledPin1Cold, OUTPUT);
    pinMode(ledPin2Good, OUTPUT);

}

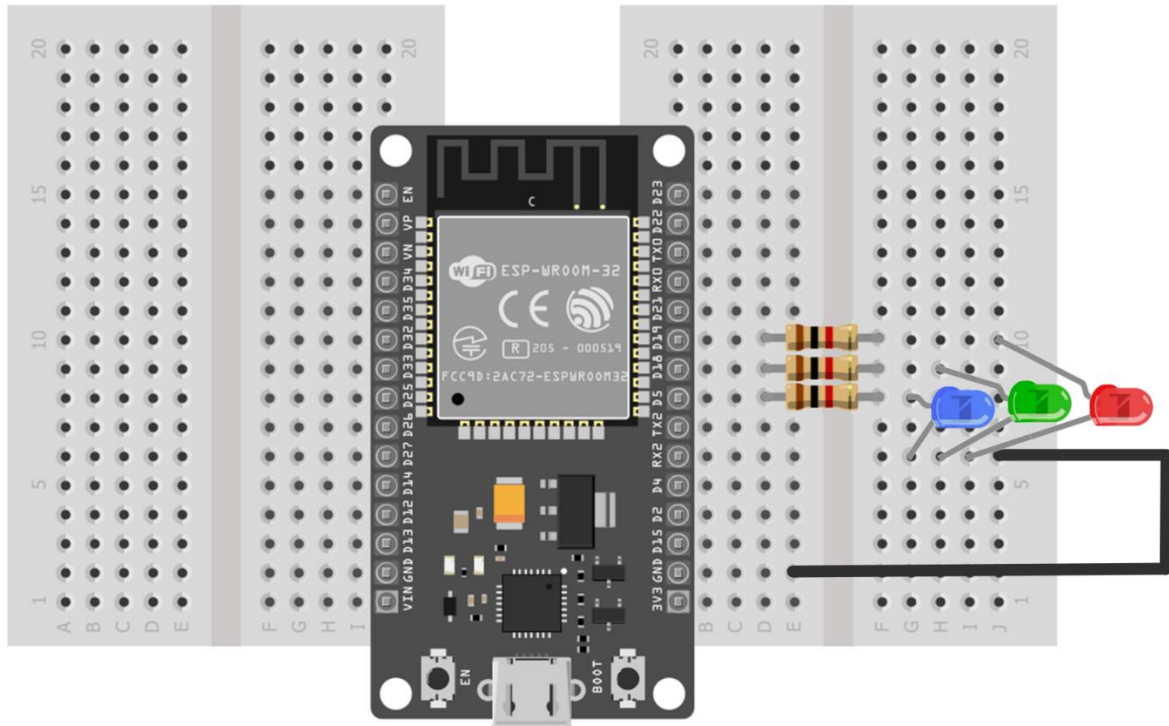
// Bucle principal del código
void loop() {

    digitalWrite(ledPin1Cold, LOW); // Apagamos el LED Azul
    digitalWrite(ledPin2Good, LOW); // Apagamos el LED Verde
    delay(1000); // Esperamos 1 segundo = 1000 milisegundos
    digitalWrite(ledPin1Cold, HIGH); // Encendemos el LED Azul
    digitalWrite(ledPin2Good, HIGH); // Encendemos el LED Verde
    delay(1000); // Esperamos 1 segundo

}
```

Como puedes ver, el objetivo del código es el mismo: encender y apagar, esta vez, ambos LED cambiando de estado cada segundo. Si el LED Azul y el Verde cambian de estado correctamente, podemos dar por válido el montaje actual y continuar con el siguiente paso.

Para terminar de montar los LEDs de colores que necesitamos para cumplir con el objetivo establecido con esta parte del Demostrador, necesitamos conectar el último LED de color Rojo en el **pin D19**. Procede a montar el diodo LED y la última resistencia correspondiente, como se muestra en la siguiente imagen.



Comprobaremos que está correcto realizando la misma programación que en los casos anteriores, generando un parpadeo, ahora, en los tres LEDs de colores. El código para ello es el que sigue.

```
#define ledPin1Cold 5 // Definimos el pin donde está conectado el LED Azul
#define ledPin2Good 18 // Definimos el pin donde está conectado el LED Verde
#define ledPin3Heat 19 // Definimos el pin donde está conectado el LED Rojo

// Bloque de inicialización
void setup() {

    // Configuramos el modo de funcionamiento de los pines de los LED como salida
    pinMode(ledPin1Cold, OUTPUT);
    pinMode(ledPin2Good, OUTPUT);
    pinMode(ledPin3Heat, OUTPUT);

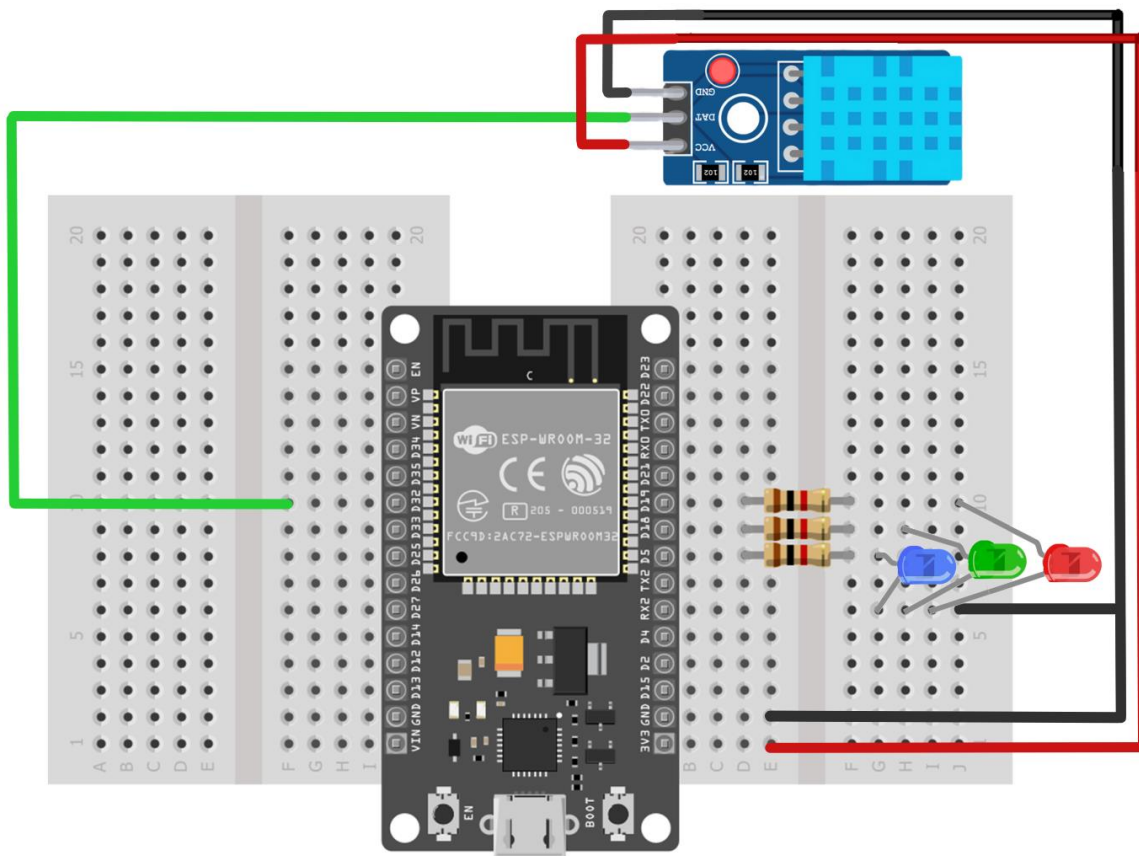
}

// Bucle principal del código
void loop() {
```

```
digitalWrite(ledPin1Cold, LOW); // Apagamos el LED Azul
digitalWrite(ledPin2Good, LOW); // Apagamos el LED Verde
digitalWrite(ledPin3Heat, LOW); // Apagamos el LED Rojo
delay(1000); // Esperamos 1 segundo = 1000 milisegundos
digitalWrite(ledPin1Cold, HIGH); // Encendemos el LED Azul
digitalWrite(ledPin2Good, HIGH); // Encendemos el LED Verde
digitalWrite(ledPin3Heat, HIGH); // Encendemos el LED Rojo
delay(1000); // Esperamos 1 segundo

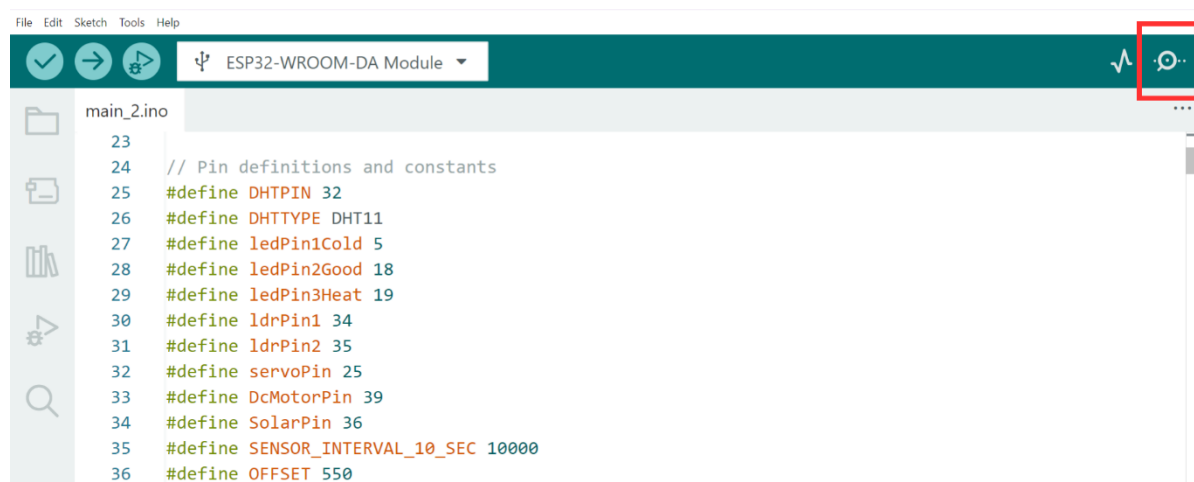
}
```

El próximo paso es agregar y conectar el sensor DHT11 en el **pin D32**, ya sea como un componente individual o en un módulo. A continuación se presenta el montaje realizado hasta este momento junto con las conexiones del sensor DHT11.



Nota: Comprobar que las conexiones GND y VCC del sensor DHT11 se corresponden con la descripción de los pines reales del componente, ya que puede ser diferente a la que aparece en el diagrama.

Pasamos a comprobar el correcto funcionamiento del sensor DHT-11. A continuación, dispones de un código de prueba de todo el montaje realizado para esta parte del montaje. Carga el programa y verás cómo, además de que los LEDs de colores parpadean como han hecho hasta ahora, la placa con ESP32 enviará por el Monitor Serial (puedes ver el botón donde abrirlo en la siguiente imagen) cada par de segundos las medidas obtenidas de temperatura ambiental (en °C) y humedad relativa del aire (en %).



```
#include <DHT.h> // Incluimos la librería necesaria para controlar de DHT

#define ledPin1Cold 5 // Definimos el pin donde está conectado el LED Azul
#define ledPin2Good 18 // Definimos el pin donde está conectado el LED Verde
#define ledPin3Heat 19 // Definimos el pin donde está conectado el LED Rojo
#define DHTPIN 32 // Definimos el pin donde está conectado el DHT
#define DHTTYPE DHT11 // Definimos el tipo de sensor DHT que vamos a utilizar

// Creamos un objeto de tipo DHT con la configuración correspondiente
DHT myDHT11(DHTPIN, DHTTYPE);

// Bloque de inicialización
void setup() {

    // Configuramos el modo de funcionamiento de los pines de los LED como salida
```



```
pinMode(ledPin1Cold, OUTPUT);
pinMode(ledPin2Good, OUTPUT);
pinMode(ledPin3Heat, OUTPUT);

// Inicializamos el monitor Serial
Serial.begin(115200);
// Inicializamos el sensor DHT
myDHT11.begin();

}

// Bucle principal del código
void loop() {

    digitalWrite(ledPin1Cold, LOW); // Apagamos el LED Azul
    digitalWrite(ledPin2Good, LOW); // Apagamos el LED Verde
    digitalWrite(ledPin3Heat, LOW); // Apagamos el LED Rojo
    delay(1000); // Esperamos 1 segundo = 1000 milisegundos
    digitalWrite(ledPin1Cold, HIGH); // Encendemos el LED Azul
    digitalWrite(ledPin2Good, HIGH); // Encendemos el LED Verde
    digitalWrite(ledPin3Heat, HIGH); // Encendemos el LED Rojo
    delay(1000); // Esperamos 1 segundo

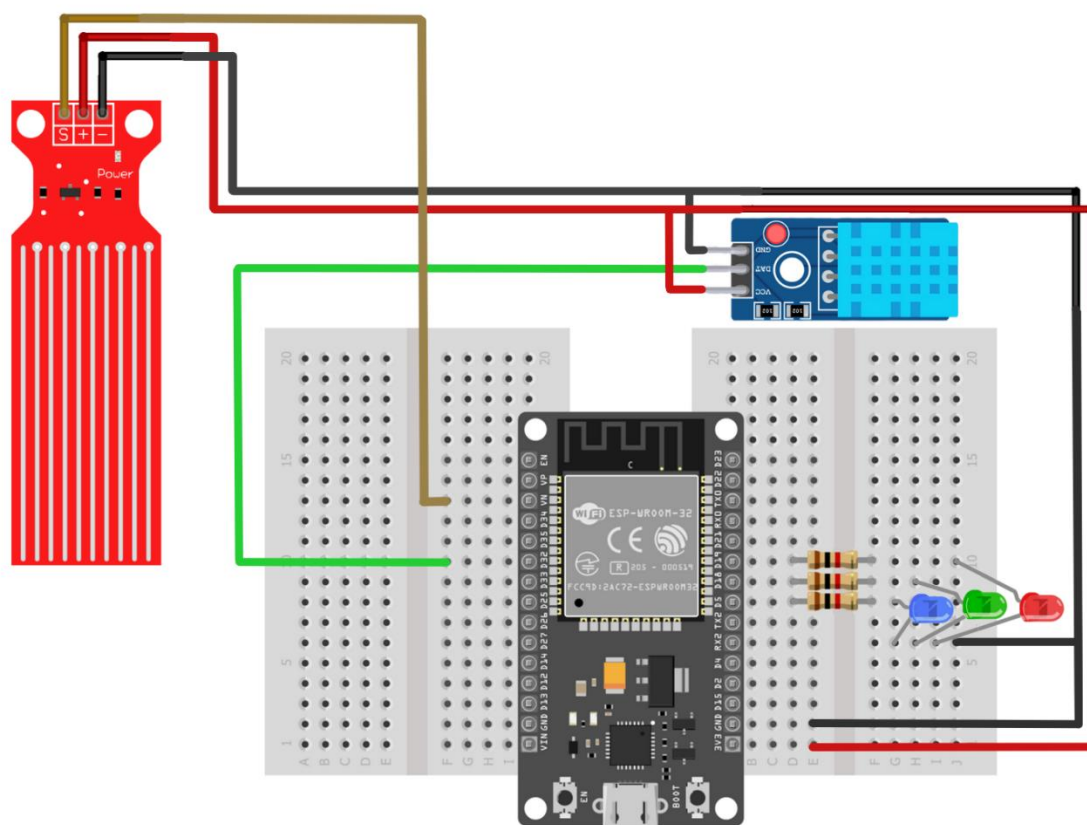
    //Obtenemos las medidas del sensor DHT
    float temperatura = myDHT11.readTemperature();
    float humedad = myDHT11.readHumidity();

    // Mostramos los valores por monitor Serial
    Serial.print("Temperatura: ");
    Serial.println(temperatura);
    Serial.print("Humedad: ");
    Serial.println(humedad);

}
```

Con el monitor abierto, comprueba que el desplegable que se ubica más a la derecha sobre el propio Monitor tenga seleccionado el valor “115200”. Ahora podrás ver los datos que envía el ESP32 tomados del DHT-11 de manera constante y verificar que el montaje hasta este punto funciona correctamente.

El siguiente componente que se va a añadir va a ser el módulo de sensor de nivel de agua. Conectamos el sensor en el **pin VN**. En la siguiente imagen se ve como quedaría el montaje:



Ahora vamos a verificar el funcionamiento del sensor de nivel de agua leyendo y mostrando por el monitor serial constantemente el valor medido del sensor:

```
//Water sensor calibration
const int WaterSensorPin = 39; // Analog Value from VN PIN

void setup() {
  Serial.begin(115200);
}

void loop() {
  int sensorValue; // Variable to store sensor values
  sensorValue = analogRead(WaterSensorPin); // Read the value from the sensor
  Serial.println(sensorValue); // Print value in serial monitor
  delay(100);
}
```



1. 2. 2. Montaje electrónico (Parte Solar)

Comenzaremos realizando el montaje del componente solar. Para realizar este montaje completo necesitarás el siguiente material:



1 x Servomotor SG-90

Utilizado para mover el Demostrador solar y orientarlo hacia la luz solar.



2 x Fotorresistencias independientes

Empleados para medir el nivel de luminosidad recibida.



2 x Resistencias 1k ohm

Utilizadas solamente junto con las LDR independientes para montar un partidor de tensión controlado por intensidad lumínica.



9 x Cable dupont (Macho-Macho) 10cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



5 x Cable dupont (Macho-Macho) 20cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



3 x Cable dupont (Macho-Hembra) 10cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



4 x Cable dupont (Macho-Hembra) 20cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



1 x Detector de tensión

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



1 x Panel solar

Para comprobar el funcionamiento de los nuevos elementos añadidos al montaje, vamos a emplear un código que solamente tome la medida de la LDR para encender el LED Rojo con mayor o menor intensidad según la luminosidad medida por la fotorresistencia. Puedes ver el código a cargar a continuación.

```
#define ldrPin1 34 // Definimos el pin donde está conectado la LDR

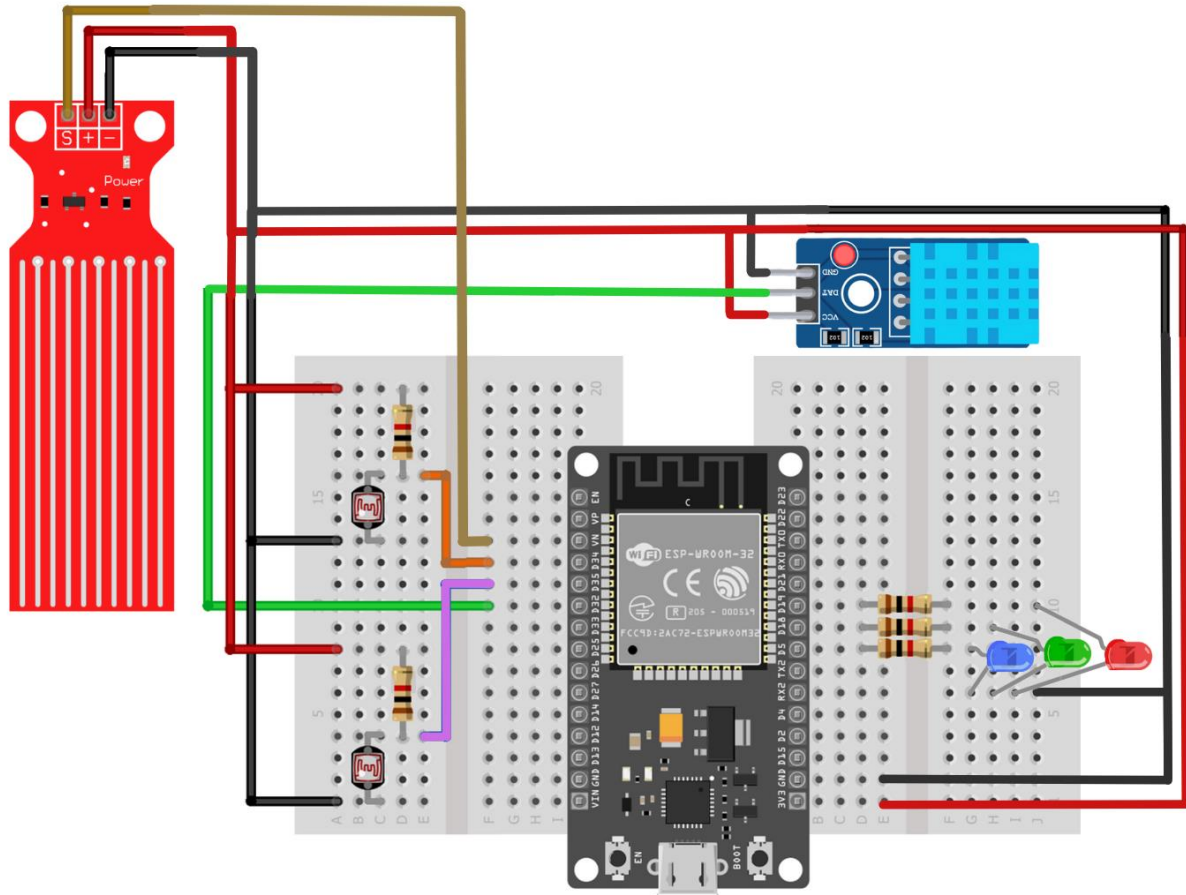
// Bloque de inicialización
void setup() {

    // Inicializamos el monitor serial
    Serial.begin(115200);
}

// Bucle principal del código
void loop() {

    // Tomamos la lectura del pin donde se encuentra la LDR
    int lectura_ldr_1 = analogRead(ldrPin1);
    // Mostramos el valor por el monitor Serial
    Serial.print("Luminosidad 1: ");
    Serial.println(lectura_ldr_1);
}
```

A continuación se debe añadir la segunda LDR con su resistencia. Para obtener el valor de la lectura de este sensor se va a utilizar la salida digital, como en el caso anterior, pero esta se va a conectar al **pin D35**. A continuación se puede ver el montaje con ambas LDR para la parte actual del Demostrador.



Comprobemos que se ha conectado correctamente empleando el siguiente código, que es muy similar al anterior pero mostrando el valor de la medida de ambas fotorresistencias.

```
#define ldrPin1 34 // Definimos el pin donde está conectado la primera LDR
#define ldrPin2 35 // Definimos el pin donde está conectado la segunda LDR

// Bloque de inicialización
void setup() {

    // Inicializamos el monitor serial
    Serial.begin(115200);

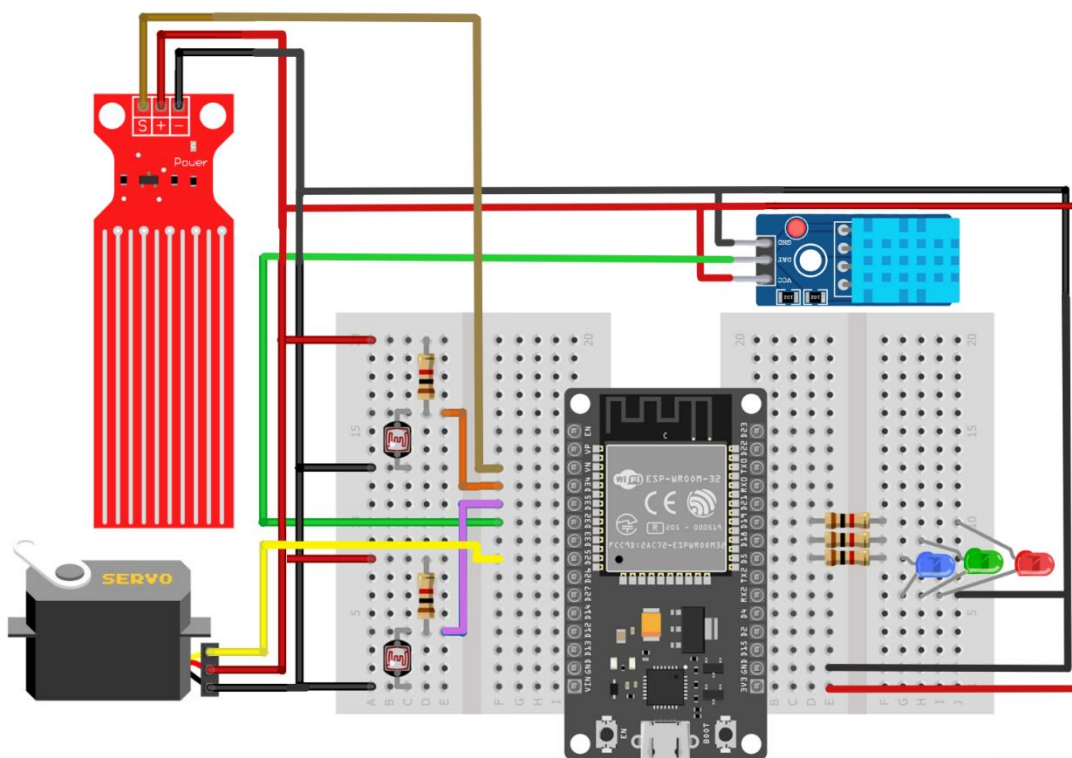
}
```

```
// Bucle principal del código
void loop() {

    // Tomamos las lecturas de los pines donde se encuentran las LDR
    int lectura_ldr_1 = analogRead(ldrPin1);
    int lectura_ldr_2 = analogRead(ldrPin2);
    // Mostramos el primer valor por el monitor Serial
    Serial.print("Luminosidad 1: ");
    Serial.println(lectura_ldr_1);
    // Mostramos el segundo valor por el monitor Serial
    Serial.print("Luminosidad 2: ");
    Serial.println(lectura_ldr_2);

}
```

El siguiente componente que se va a añadir en el montaje es un servomotor. Este se va a conectar a ambas líneas de alimentación (cable rojo a la positiva y cable negro a la negativa) y el cable amarillo (que es la señal de control) se va a conectar al **pin D25**.



Comprobaremos el funcionamiento del servomotor cargando el siguiente código en la placa ESP32 que realiza un barrido moviendo el servo entre las posiciones de 0°, 90° y 180°, en bucle, una y otra vez.

```
#include <ESP32Servo.h> // Incluir la librería necesaria para controlar el servomotor

#define servoPin 25 // Definimos el pin donde está conectado el servomotor

// Creamos un objeto de tipo Servo para controlarlo
Servo myServo;

// Bloque de inicialización
void setup() {

    // Asociamos el servo que creamos al pin donde está conectado
    myServo.attach(servoPin);
    // Colocamos el servo en una posición inicial de 0°
    myServo.write(0);

}

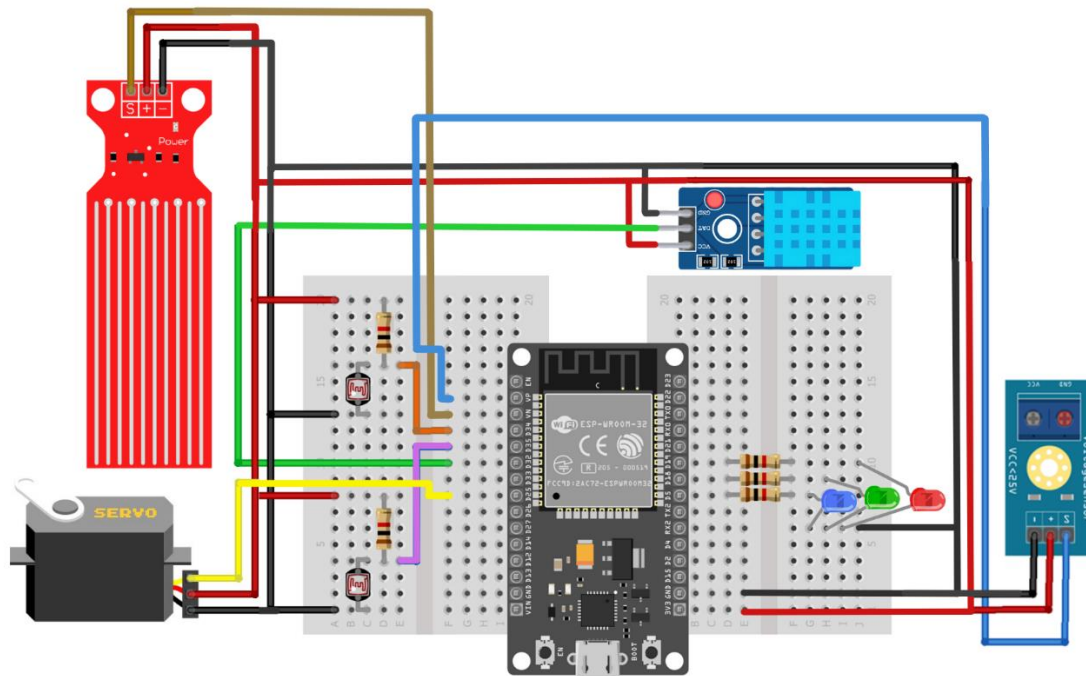
// Bucle principal del código
void loop() {

    myServo.write(0); // Movemos el servomotor a 0°
    delay(1000); // Esperamos 1 segundo = 1000 milisegundos
    myServo.write(90); // Movemos el servomotor a 90°
    delay(1000); // Esperamos 1 segundo
    myServo.write(180); // Movemos el servomotor a 180°
    delay(1000); // Esperamos 1 segundo

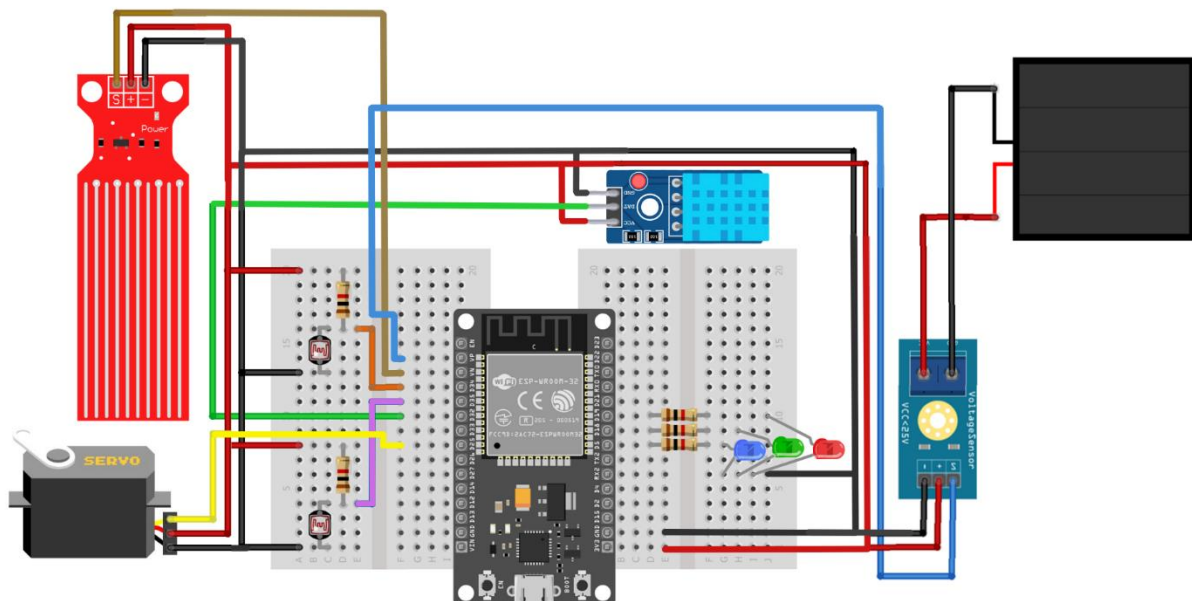
}
```

Nota: Colocar la pestaña en el servo. No es necesario usar tornillos para sujetarla.

El último componente que vamos a conectar al ESP32 será un detector de tensión. Este módulo se debe alimentar a ambas líneas de alimentación (Vcc y GND). La línea de señal se va a conectar al **pin VP**.



El último paso que falta para terminar el montaje electrónico es conectar el panel solar al detector de tensión. A continuación se puede ver cómo quedaría el montaje completo.



Vamos a comprobar el funcionamiento del detector solar completo haciendo uso del siguiente código que simplemente lee la señal del módulo detector de tensión y enciende el LED Azul si detecta voltaje.

```
#define ledPin1Cold 5 // Definimos el pin donde está conectado el LED Azul
#define detectorPin 36 // Definimos el pin donde está conectado el Detector de Tensión

// Bloque de inicialización
void setup() {

    // Configuramos el modo de funcionamiento del pin del LED como salida
    pinMode(ledPin1Cold, OUTPUT);

}

// Bucle principal del código
void loop() {

    // Tomamos la lectura del Módulo Detector
    bool tension_detectada = digitalRead(detectorPin);
    // Comprobamos si hay tensión o no
    if(tension_detectada) { // Sí hay tensión

        digitalWrite(ledPin1Cold, HIGH); // Encendemos el LED Azul

    } else { // No hay tensión

        digitalWrite(ledPin1Cold, LOW); // Apagamos el LED Azul

    }

}
```

A continuación, dejamos un código test para comprobar que todos los sensores obtienen lecturas y funcionan correctamente:

```
//include the necessary libraries
#include <ESP32Servo.h>
#include "DHT.h"

// Pin definitions and constants
#define ldrPin1 34 // set ldr 1 Analog input pin of East ldr as an integer
```

```
#define ldrPin2 35 // set ldr 2 Analog input pin of West ldr as an integer
#define DHTPIN 32
#define DHTTYPE DHT11
#define ledPin1Cold 5
#define ledPin2Good 18
#define ledPin3Heat 19
#define SolarPin 36

// Global variables
DHT dht(DHTPIN, DHTTYPE);
Servo myservo;
int valldr1 = 0 ;
int valldr2 = 0 ;
int averageLdrValue = 0;
int servopos = 90; // initial position of the Horizontal movement controlling servo motor
int tolerance = 20; // allowable tolerance setting - so solar servo motor isn't constantly
in motion
float lastMicrovolts = 0;
const int WaterSensorPin = 39; // Analog Value from VN PIN
int sensorValue; // Variable to store sensor values

void setup() {
  Serial.begin(115200);
  analogReadResolution(12); // ESP32 has a 12-bit ADC
  analogSetAttenuation(ADC_11db); // Set attenuation for higher sensitivity
  pinMode(4, INPUT);

  dht.begin(); // Start the DHT sensor

  pinMode(ledPin1Cold, OUTPUT);
  pinMode(ledPin2Good, OUTPUT);
  pinMode(ledPin3Heat, OUTPUT);

  myservo.attach(25); // attaches the servo on digital pin 2 to the horizontal movement servo
  motor

  pinMode(ldrPin1, INPUT); //set East ldr pin as an input
  pinMode(ldrPin2, INPUT); //set West ldr pin as an input
  myservo.write(servopos); // write the starting position of the horizontal movement servo
  motor
  delay(1000); // 1 second delay to allow the solar panel to move to its staring position
  before commencing solar tracking
```

```

}

void loop(){
  Serial.println("----- Measured Solar (V) -----
  ----- ");
  int solarVoltageV = analogRead ( SolarPin); //dV
  int solarVoltage = solarVoltageV * 10;

  Serial.print("Voltage of Solar Panel: ");
  Serial.println(solarVoltage);

  // Calculate voltage
  float voltage = (solarVoltage / 4095.0) * 3.3; // For 12-bit ADC (0-4095)
  Serial.print("Measured voltage (V): ");
  Serial.println(voltage); // Display the measured voltage

  // Convert to microvolts
  float microvoltsSolar = voltage * 1000000.0;
  Serial.print("Input voltage (µV): ");
  Serial.println(microvoltsSolar);

  // Apply a minimum threshold to filter noise
  const float threshold = 10.0; // Threshold in microvolts
  if (microvoltsSolar < threshold) {
    microvoltsSolar = 0;
  }

  // Check if the microvolts value has changed
  if (microvoltsSolar != lastMicrovolts) {
    Serial.print("Send voltage: ");
    Serial.print(microvoltsSolar, 2); // Display with 2 decimal places
    Serial.println(" µV");

    // Update the last microvolts value
    lastMicrovolts = microvoltsSolar;
  }

  Serial.println("----- Measured Temperature / Humidity -----
  ----- ");
  // Read values from DHT sensor
  float h = dht.readHumidity();
  float t = dht.readTemperature();

```

```
// Check if readings are valid
if (isnan(h) || isnan(t)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
}

// Print sensor values
Serial.print("Temperature: ");
Serial.print(t);
Serial.print(" °C, Humidity: ");
Serial.print(h);
Serial.print(" %");

// Example LED control based on temperature
if (t < 20) {
    digitalWrite(ledPin1Cold, HIGH);
    digitalWrite(ledPin2Good, LOW);
    digitalWrite(ledPin3Heat, LOW);
    Serial.println("Cold is HIGH");
} else if (t >= 20 && t < 25) {
    digitalWrite(ledPin1Cold, LOW);
    digitalWrite(ledPin2Good, HIGH);
    digitalWrite(ledPin3Heat, LOW);
    Serial.println("Good is HIGH");
} else {
    digitalWrite(ledPin1Cold, LOW);
    digitalWrite(ledPin2Good, LOW);
    digitalWrite(ledPin3Heat, HIGH);
    Serial.println("Heat is HIGH");
}

Serial.println("----- Measured Water Level -----");
//----- ";
int sensorValue; // Variable to store sensor values
sensorValue = analogRead(WaterSensorPin); // Read the value from the soil moisture sensor
//Check watersensor calibration first
if (sensorValue >= 2000) { //These value represents de max value given
    sensorValue = 1980; //This value would return a 99% value
}
float waterlevel = (sensorValue*100)/2000 ; //Calculation to return the %
```

```

Serial.print("Water level: "); // Print value in serial monitor
Serial.print(waterlevel); // Print value in serial monitor
Serial.println(" %"); // Print value in serial monitor

Serial.println("----- Measured LDR and Servo -----
----- ");
    // Read LDR values
    valldr1 = analogRead(ldrPin1); // read the value of ldr 1
    valldr2 = analogRead(ldrPin2); // read the value of ldr 2

    // Invert readings so low values mean darkness and high values mean light

    averageLdrValue = (valldr1 + valldr2) / 2;

    // Print readings for debugging
    Serial.print("LDR 1: ");
    Serial.print(valldr1);
    Serial.print(" | LDR 2: ");
    Serial.println(valldr2);
    Serial.print("Average LDR: ");
    Serial.println(averageLdrValue);

    if((abs(valldr1 - valldr2) <= tolerance) || (abs(valldr2 - valldr1) <= tolerance)) {
    //no servo motor horizontal movement will take place if the ldr value is within the
    allowable tolerance
    }else {
    if(valldr1 > valldr2) // if ldr1 senses more light than ldr2
    {
    servopos = servopos+10; // decrement the 90 degree position of the horizontal servo motor
    - this will move the panel position Eastward
    }
    if(valldr1 < valldr2) // if ldr2 senses more light than ldr1
    {
    servopos = servopos-10; // increment the 90 degree position of the horizontal motor - this
    will move the panel position Westward
    }
    }
    if(servopos > 180) {servopos = 180;} // reset the horizontal position of the motor to 180
    if it tries to move past this point
    if(servopos < 0) {servopos = 0;} // reset the horizontal position of the motor to 0 if it
    tries to move past this point
    myservo.write(servopos); // write the starting position to the horizontal motor

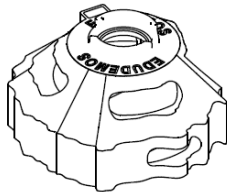
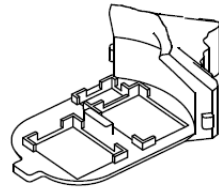
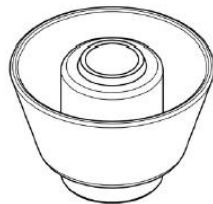
```

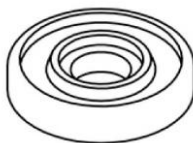
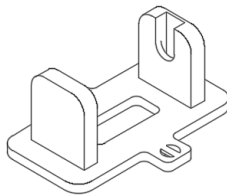
```
// Print the servo angle  
Serial.print("Servo angle: ");  
Serial.print(servopos);  
delay(5000);  
}
```



1. 2. 3. Montaje Mecánico del Demostrador de Agua + Solar

Los elementos que componen este demostrador son los siguientes:

Elemento	Cantidad	Descripción	Imagen
Base	1	Es la parte superior que asegura los componentes móviles, como las palas y el eje del motor. Proporciona estabilidad y permite que el sistema funcione suavemente, facilitando la correcta transferencia de energía desde las palas al generador.	
Cajón	1	Es la base que conecta las diferentes secciones del dispositivo, proporcionando soporte estructural y asegurando una correcta alineación entre el motor, las palas y otros componentes esenciales para su funcionamiento.	
Contenedor de agua	1	Esta parte es capaz de almacenar el agua que cae en él.	
Hoja	8	Hoja que permite que los líquidos entren en el contenedor de agua	

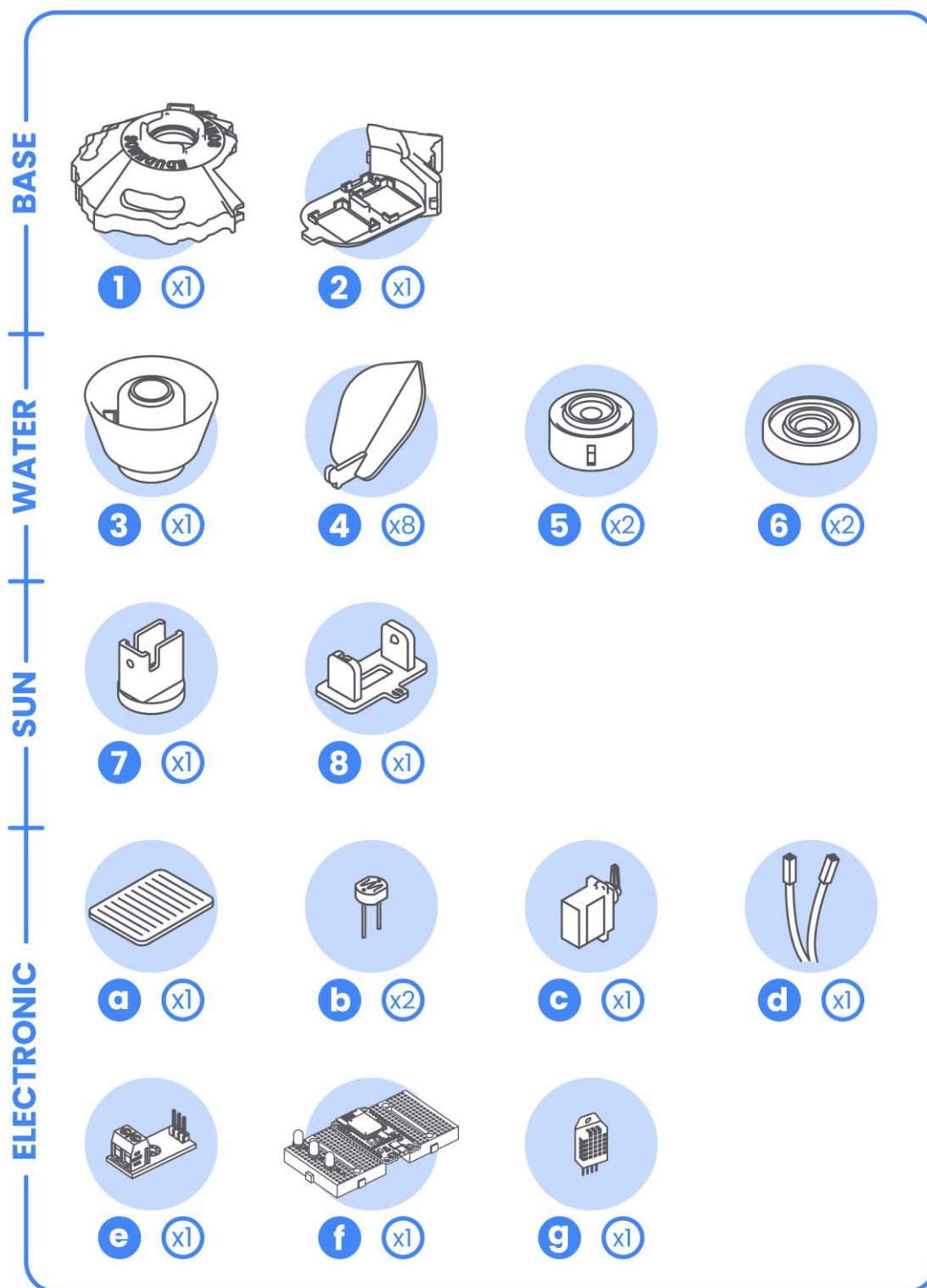
Disco Pequeño	3	Es un componente estático que no se mueve. Su función principal es actuar como parte del soporte estructural del dispositivo, ayudando a mantener la correcta alineación y estabilidad de los elementos.	
Disco Estático	2	Disco que se mantiene fija para tapar y mantener la estabilidad de la estructura.	
Placa Sol	1	Parte que permite el movimiento del panel solar para su correcta orientación	
Disco Solar	1	Disco que permite conectar el girador solar con un servomotor instalado y juntarlo con el resto de la estructura.	

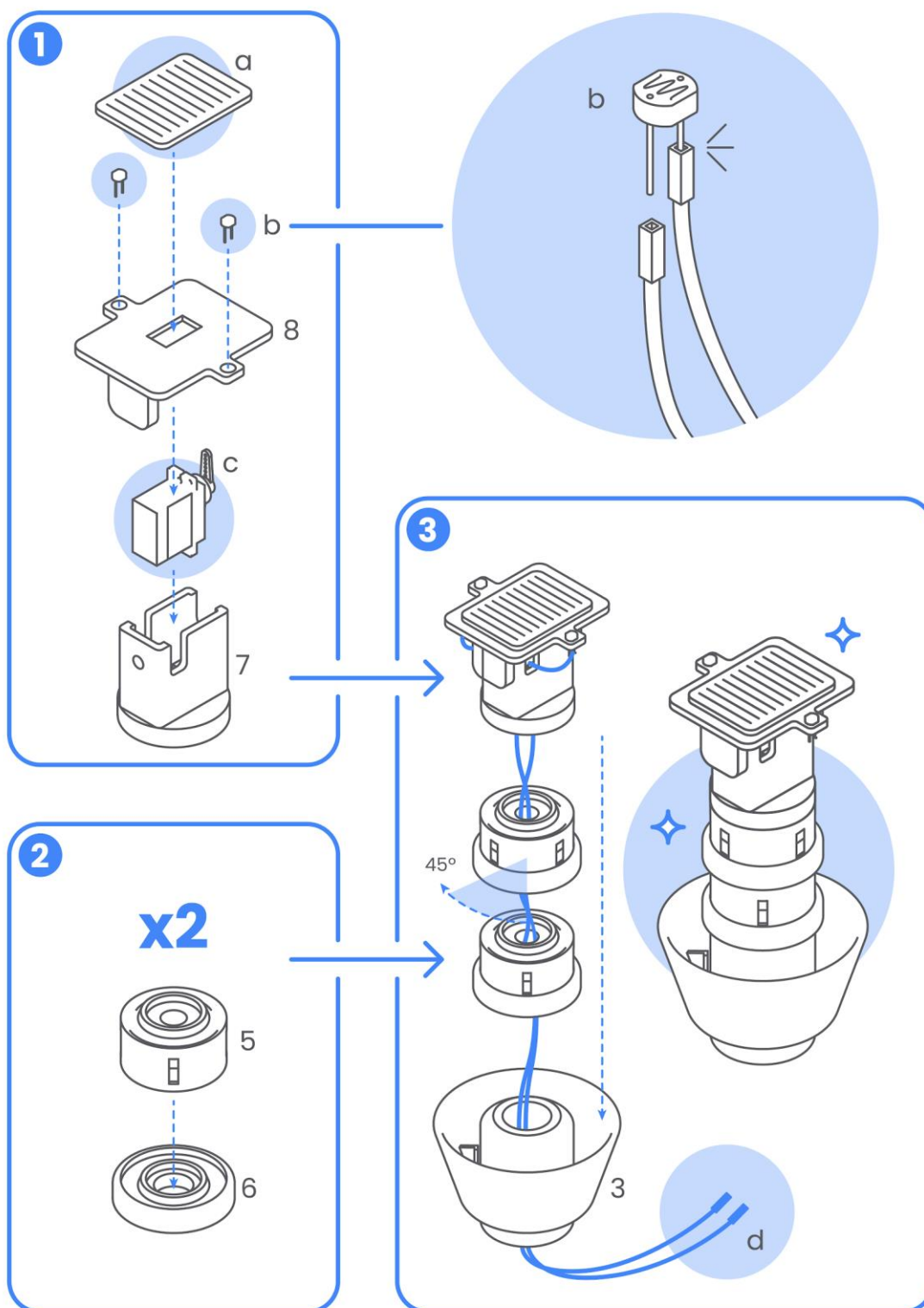
A continuación se muestra cómo se realiza el montaje del demostrador:

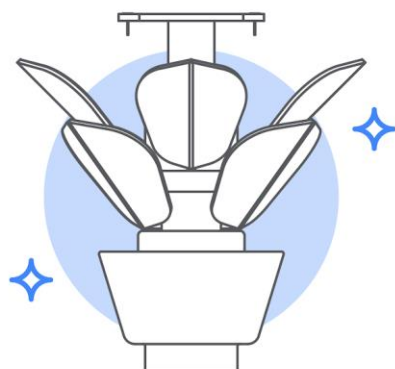
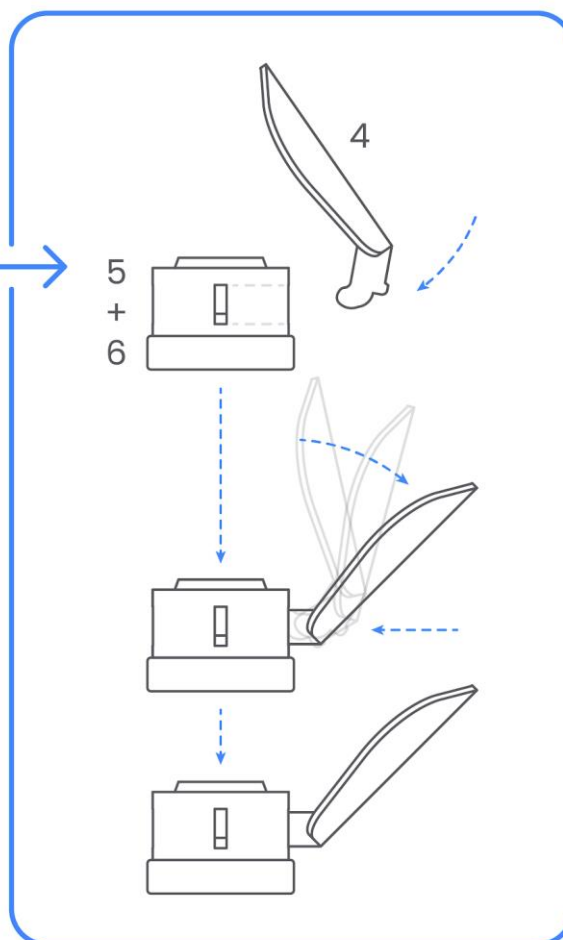
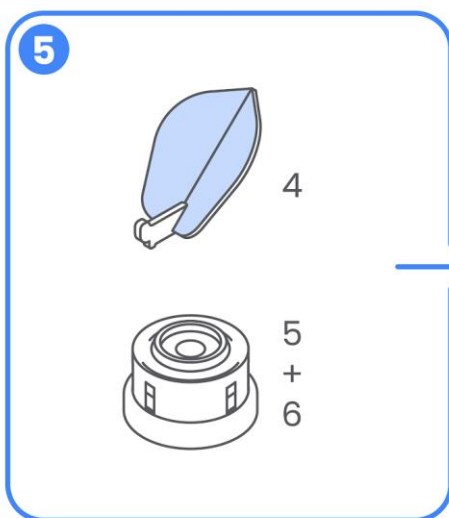
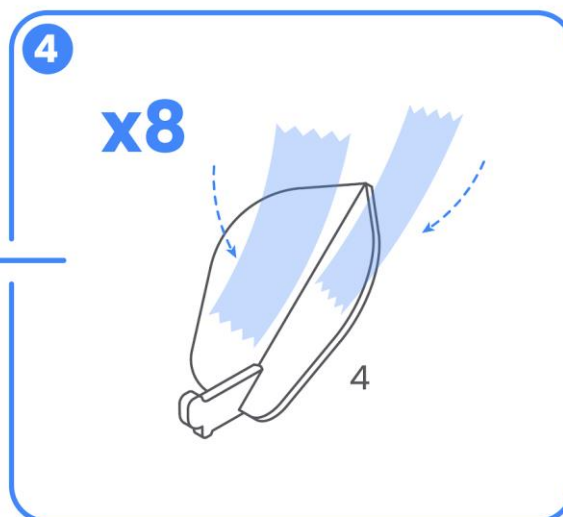
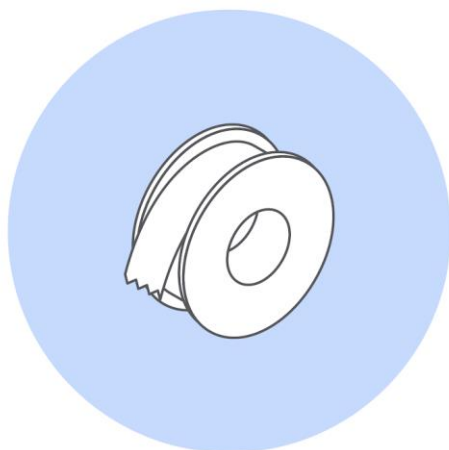
Nota: La cinta de aluminio se utiliza en la superficie de las hojas para optimizar la recolección de agua. La cinta de doble cara se utiliza en la parte trasera del panel solar para pegarlo en la pieza del "placa sol".

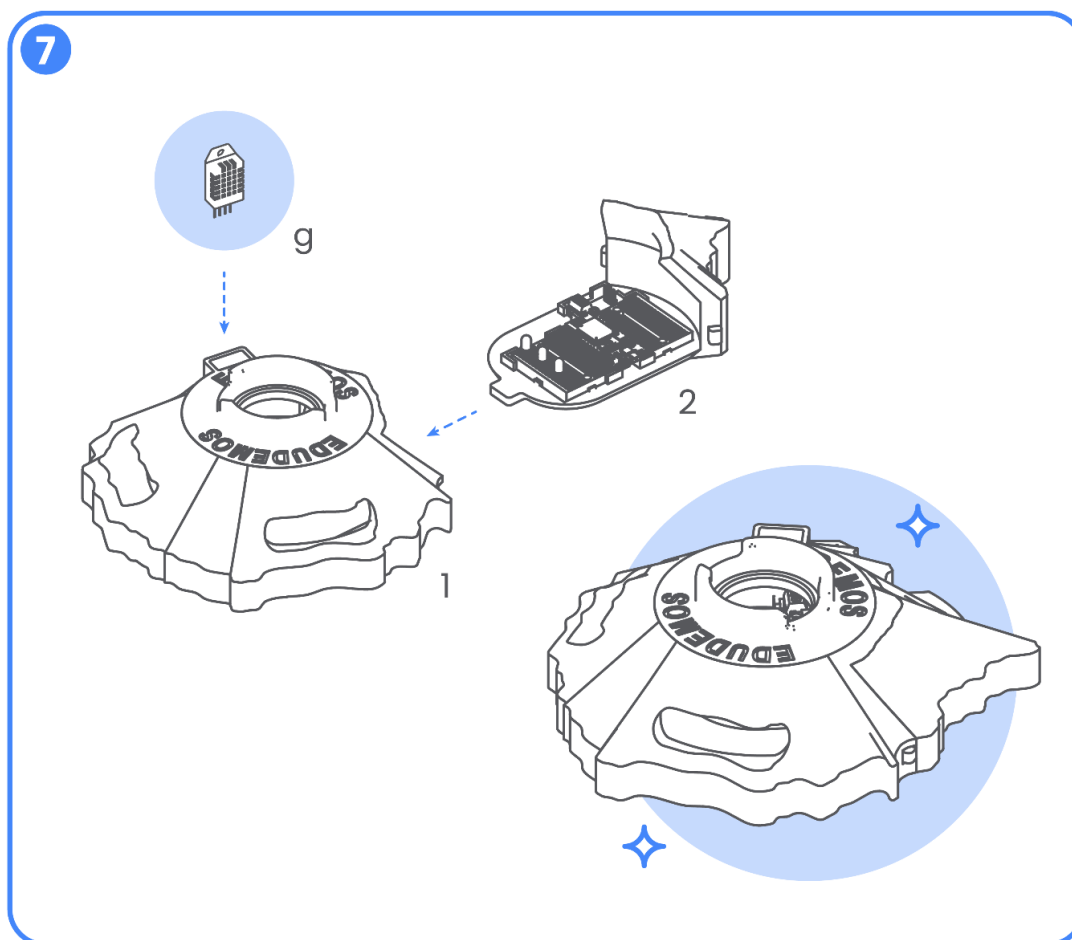
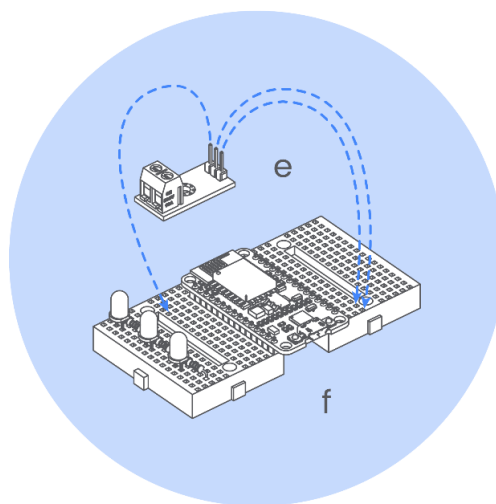
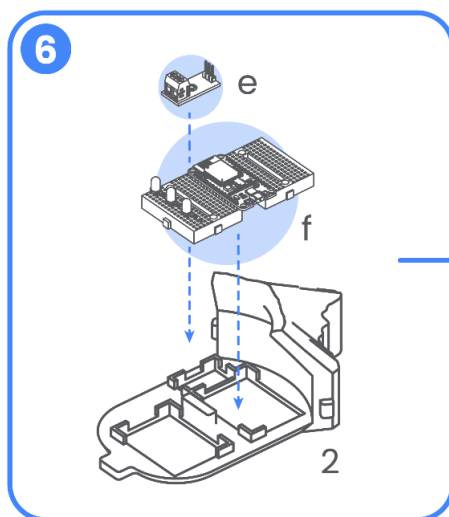
El sensor de nivel de agua va colocado en unas ranuras dentro de la pieza "contenedor de agua".

Al colocar los componentes electrónicos en el montaje mecánico, es necesario usar cables adicionales para alargar las conexiones de las LDR y de la placa solar.

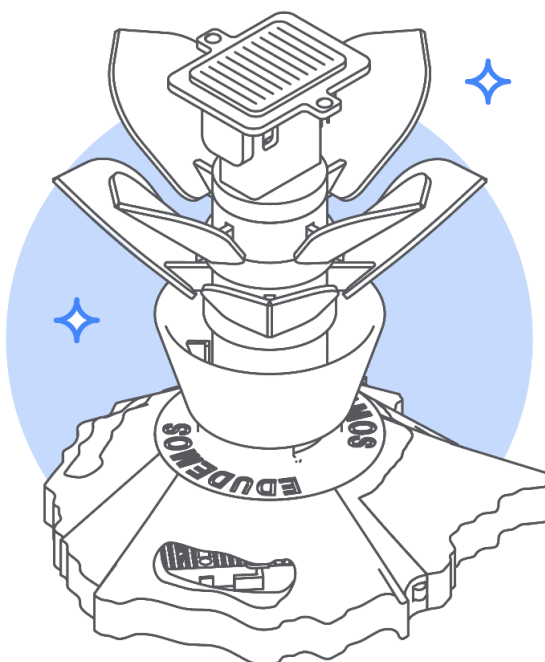
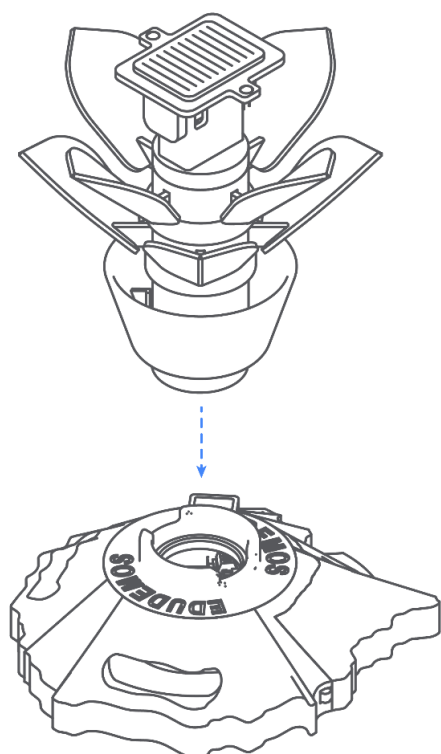








8



1. 3. Montaje del Demostrador de viento

1. 3. 1. Montaje electrónico

Comenzaremos realizando el montaje del Demostrador de viento. Para realizar este montaje completo necesitarás el siguiente material:



1 x Motor de Corriente Continua de 5V

Empleado para generar voltaje con la fuerza del viento.



1 x Condensador de 100nF / 1uF

Utilizados para eliminar ruidos indeseados generados por el Motor de CC.



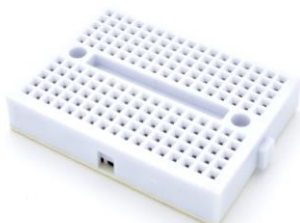
2 x Resistencias de 1 kΩ

Utilizados en el divisor de tensión que permite tomar la lectura de tensión generada por el Motor de CC.



6 x Cable dupont (Macho-Macho) 10cm

Necesarios para realizar las conexiones entre los diferentes componentes en la protoboard.



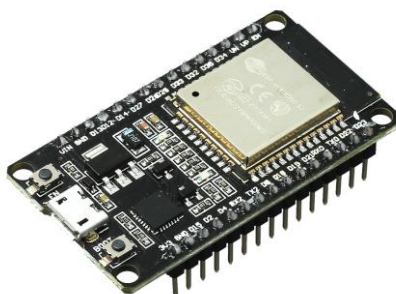
2 x Mini Breadboard

Componente necesario para conectar la placa microcontroladora con los diferentes componentes



Soldador de estaño

sólo si el motor no tiene cables incluidos



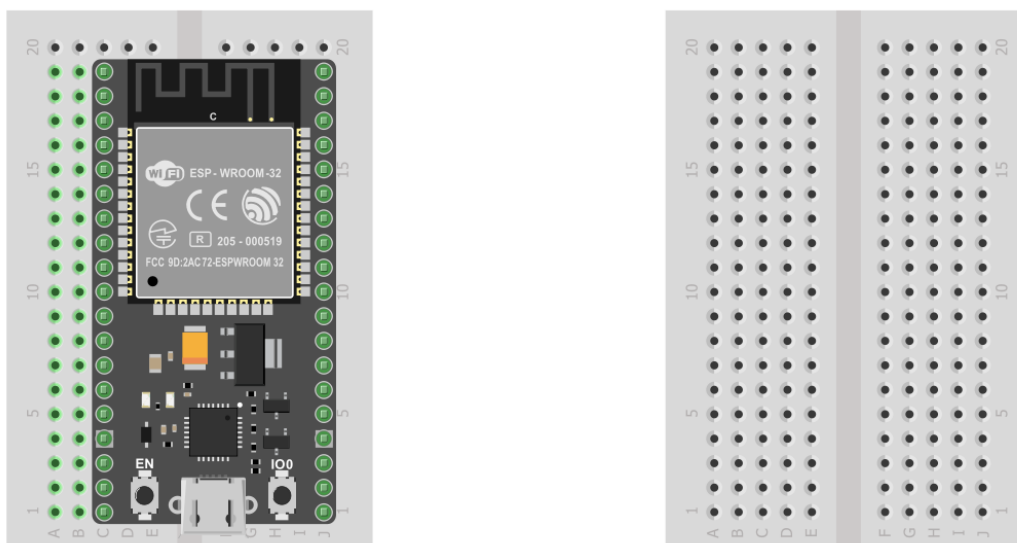
1 x ESP32 DEVKIT V1 (30 pins)

Encargado de ejecutar la programación que controla el funcionamiento de la electrónica.

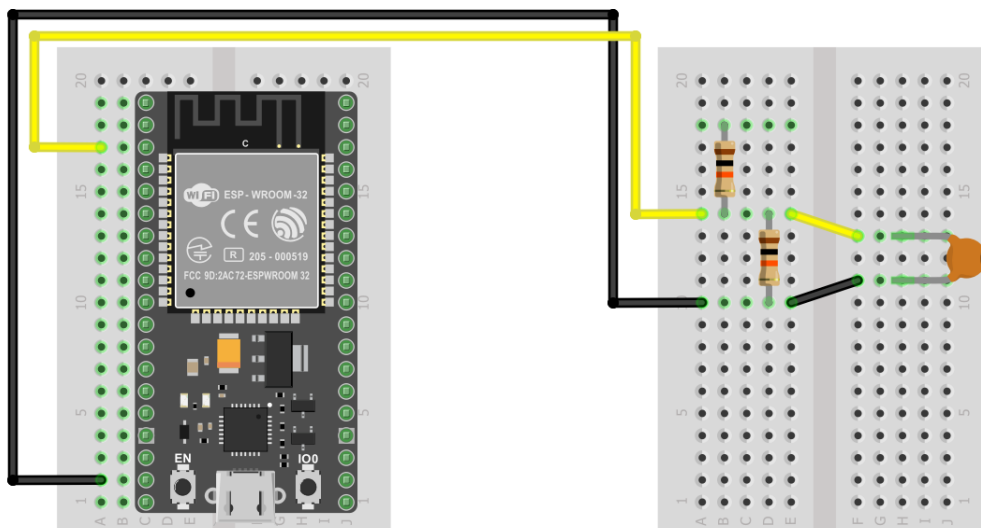
El objetivo del montaje será obtener la medida de la cantidad de voltaje generado por el motor, que irá unido a la estructura mecánica que girará por efecto de la fuerza del viento. Para poder realizar la medición con el ESP32 de la tensión obtenida por el movimiento de las aspas del Demostrador, debemos montar un partidor de tensión que

reduzca el nivel máximo de tensión a obtener (5V) a uno que sea apto para el ESP32 (que trabaja a 3'3V). Con esto podremos obtener cuánto voltaje se está generando por efecto del viento.

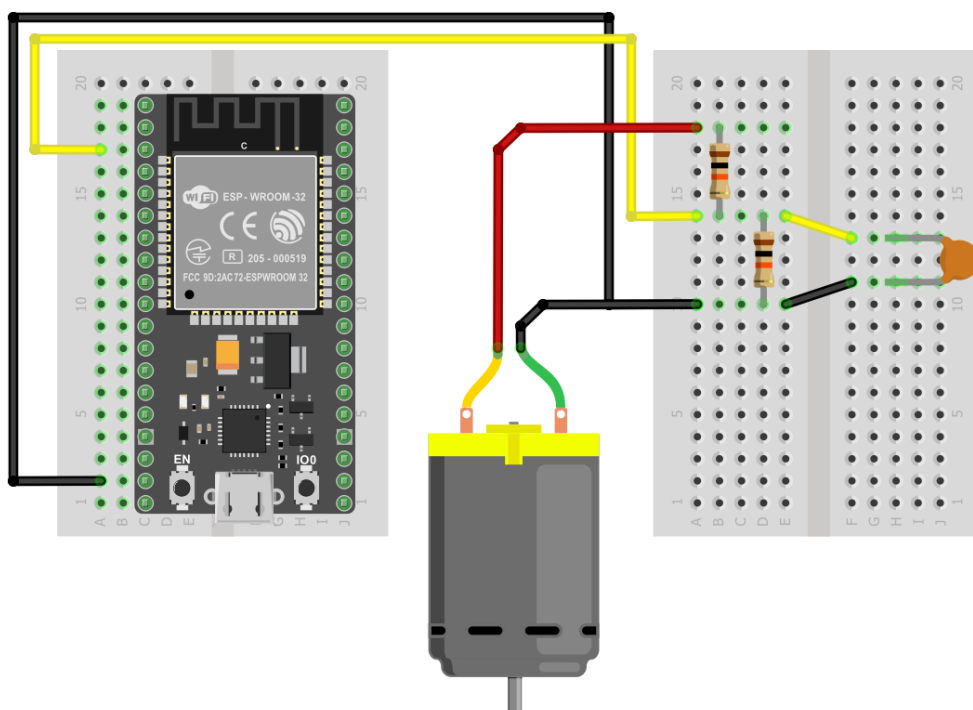
Para este nuevo montaje, será necesario crear un circuito desde el principio. Comenzaremos colocando el ESP32 en dos mini protoboards. A continuación, se muestra el montaje correspondiente.



A continuación, se procederá a montar un divisor de tensión. El punto intermedio de este divisor se conectará al pin **D39 (VN)**. Además, es necesario conectar un condensador entre el terminal de este pin y GND, que servirá de filtro para las variaciones indeseadas en la medida, tal y como se muestra en la imagen siguiente.



El último paso en el montaje electrónico de este demostrador es conectar el motor al conjunto anterior, de manera que uno de sus terminales se une a GND y el otro terminal a la entrada del divisor de tensión montado antes. A continuación, se presenta este montaje terminado.

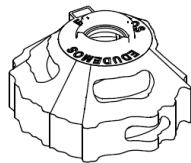
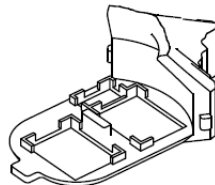


Vamos a comprobar que el montaje de este Demostrador funciona correctamente haciendo uso del siguiente fragmento de código en el que solamente leemos la salida del divisor de tensión que está unida al ESP32. Girando el rotor del motor, veremos como la medición va dando diferentes valores a través del Monitor Serial.

```
// Bloque de inicialización
void setup() {
    // Inicializamos el Monitor Serial
    Serial.begin(115200);
}
// Bucle principal del código
void loop() {
    // Tomamos la medida del pin que va a la salida del partidor de tensión
    int medicion = analogRead(39);
    // Mostramos la medida por el monitor serial
    Serial.println(medicion);
}
```

1.3.2. Montaje Mecánico

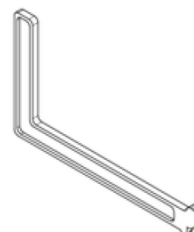
Los elementos que componen este demostrador son los siguientes:

Elemento	Cantidad	Descripción	Imagen
Base	1	Es la parte superior que asegura los componentes móviles, como las palas y el eje del motor. Proporciona estabilidad y permite que el sistema funcione suavemente, facilitando la correcta transferencia de energía desde las palas al generador.	
Cajón	1	Es la base que conecta las diferentes secciones del dispositivo, proporcionando soporte estructural y asegurando una correcta alineación entre el motor, las palas y otros componentes esenciales para su funcionamiento.	
Soporte Motor	1	Esta pieza se encargará de sostener y brindar apoyo al motor, cuyo eje estará conectado al disco de rodamiento. Este diseño permitirá la conexión de los pines de salida, que emitirán señales indicando la presencia de movimiento.	

Hoja Grande en L

6

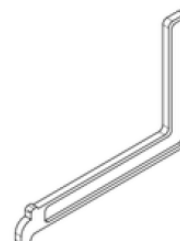
Es una hoja fija que no se mueve. Está diseñada para formar parte de la estructura estática del demostrador, contribuyendo a la estabilidad y el equilibrio del sistema en su conjunto.



Hoja Pequeña en L

6

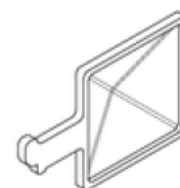
Es una hoja fija que no se mueve, de menor tamaño que la pieza anterior. Está diseñada para formar parte de la estructura estática del demostrador, contribuyendo a la estabilidad y el equilibrio del sistema en su conjunto.



Pala Cuadrada

8

Es una de las palas móviles clave que gira cuando es impulsada por el viento. Su diseño cuadrado permite capturar eficientemente la energía cinética del viento y convertirla en movimiento rotatorio.



Disco grande

2

Es un componente clave que actúa como soporte estructural para las palas móviles. Este disco está diseñado para girar cuando las palas capturan el viento, transfiriendo el movimiento rotatorio hacia el eje central del sistema.



Disco Pequeño

3

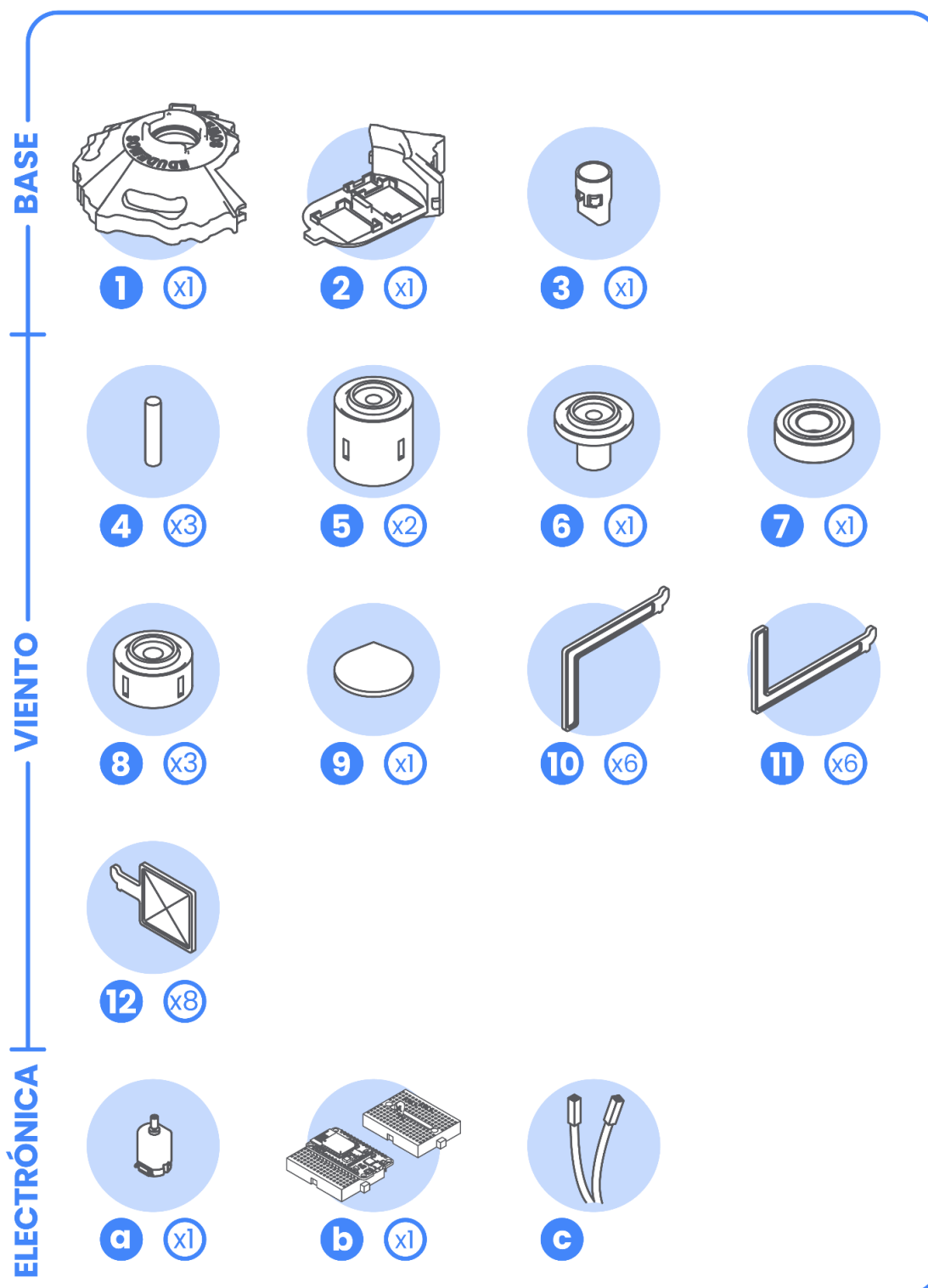
Es un componente estático que no se mueve. Su función principal es actuar como parte del soporte estructural del dispositivo, ayudando a mantener la correcta alineación y estabilidad.

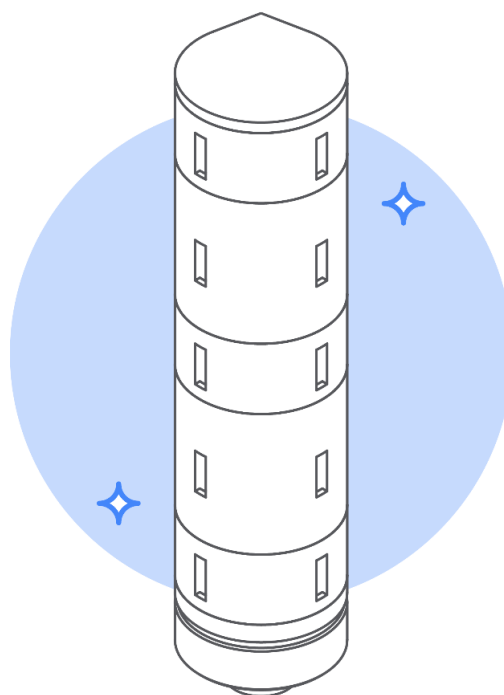
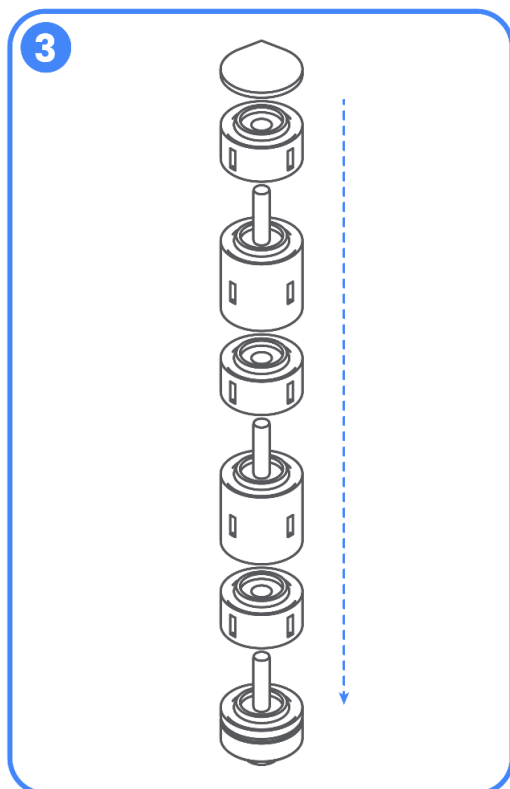
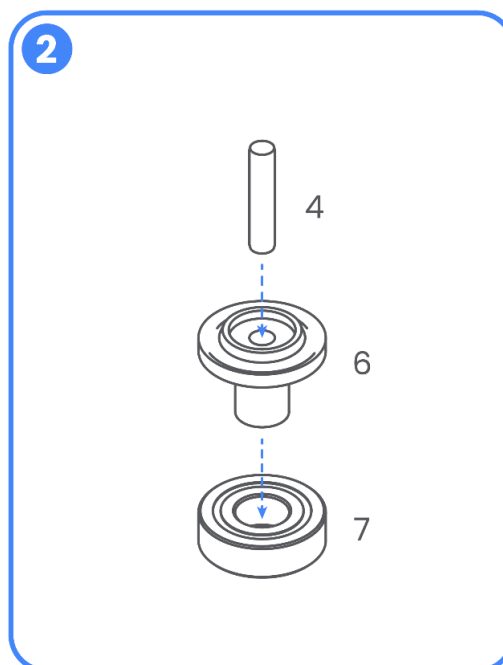
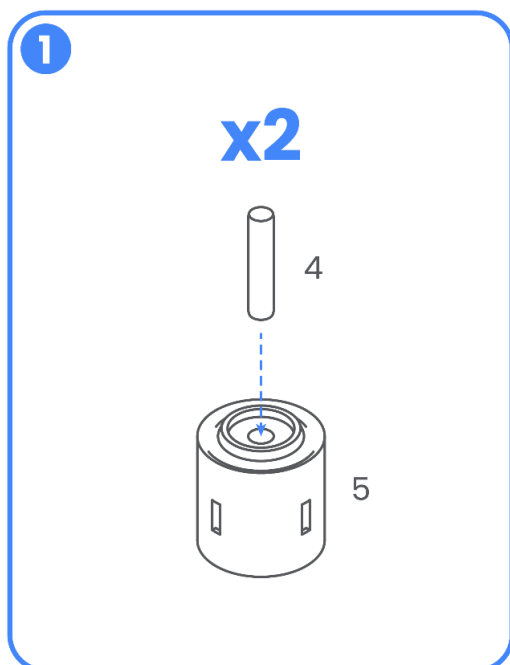


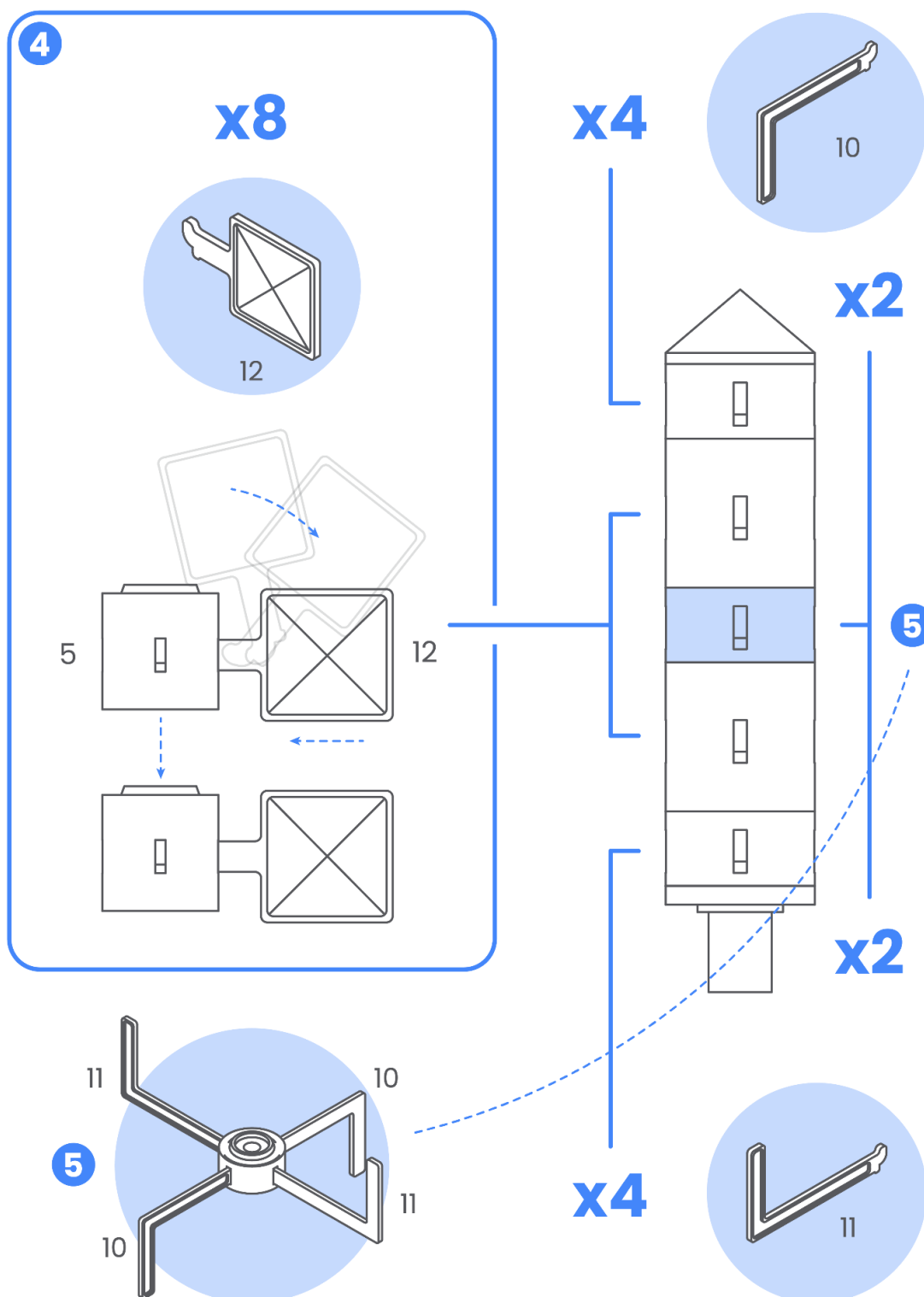
Disco rodamiento	1	de los elementos móviles, como las hojas y los discos grandes.	
		Es un componente crucial que se utiliza para reducir la fricción entre las partes móviles del sistema. Este disco actúa como un soporte rotatorio que permite que los componentes, como el Disco Grande y las palas, giren de manera suave y eficiente alrededor del eje central.	
		Es un componente estructural que se coloca en la parte superior del conjunto, asegurando que todos los elementos internos estén protegidos y correctamente alineados.	
		Es un componente mecánico que permite que los elementos giratorios, como los discos y las palas, se muevan con mínima fricción.	
Tapa	1		
Rodamiento	1		
Espigas Madera	3	Es un componente estructural utilizado para conectar y asegurar las diferentes piezas de madera del dispositivo.	

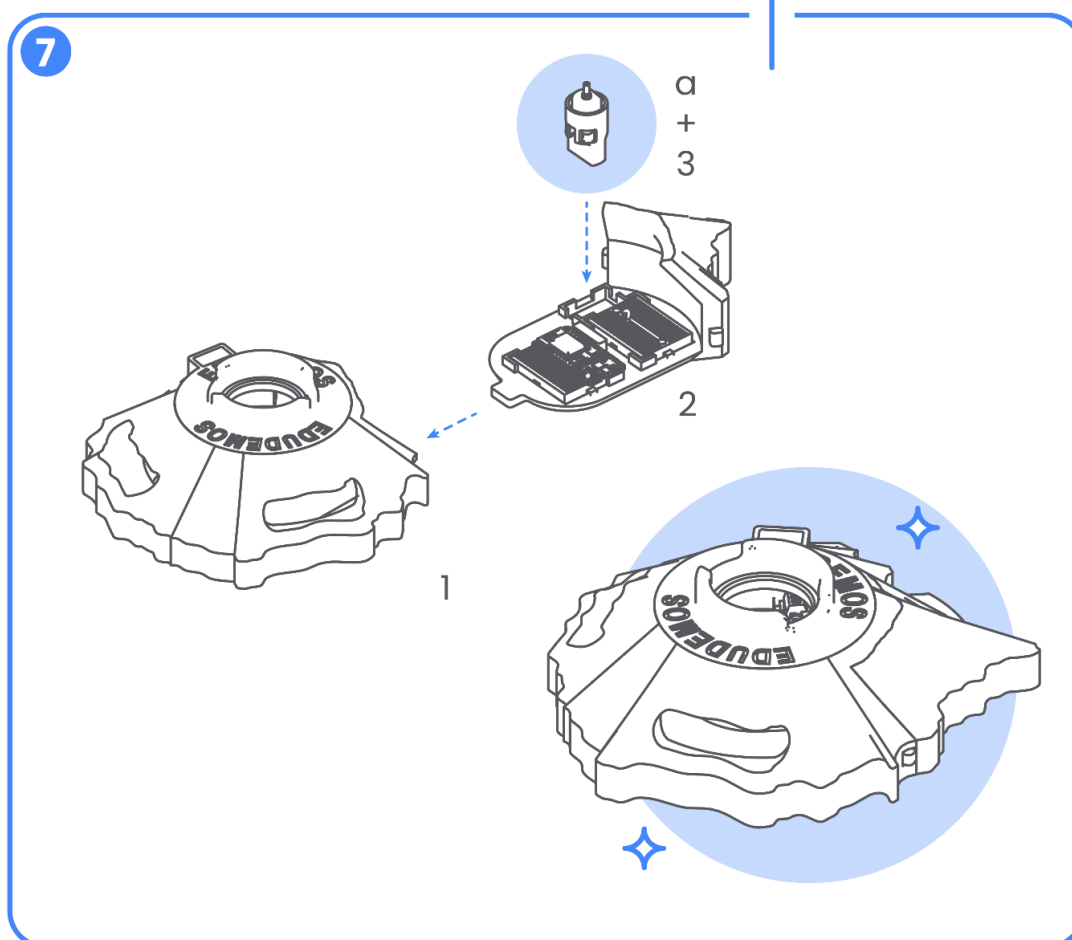
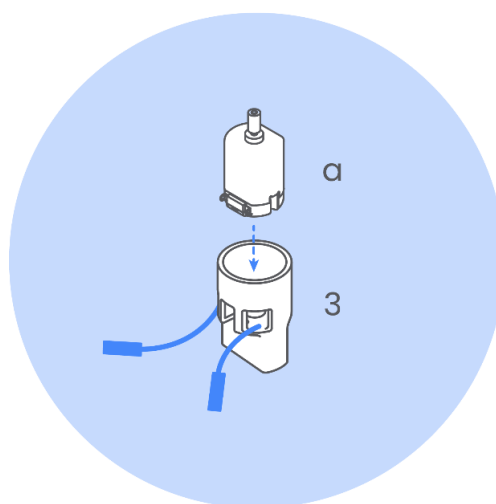
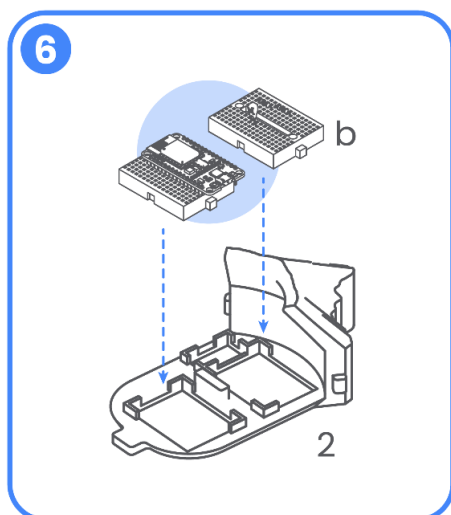
A continuación se muestra cómo se realiza el montaje del demostrador:

Nota: Es necesario insertar el engranaje de 10 dientes en el eje del motor DC antes de comenzar con el montaje del demostrador.

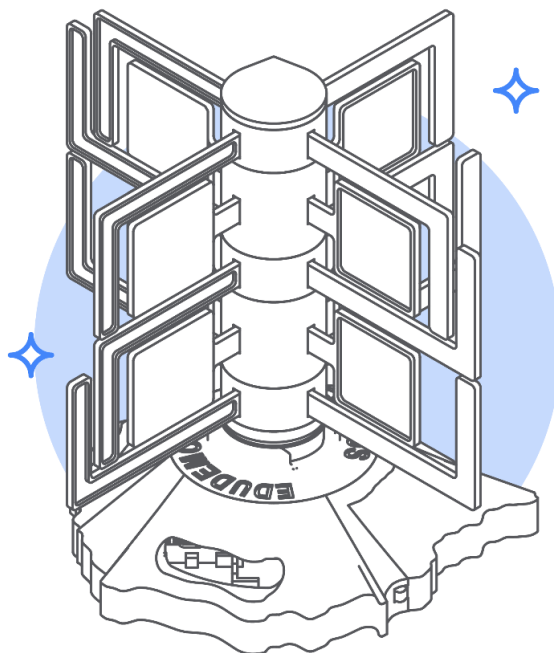
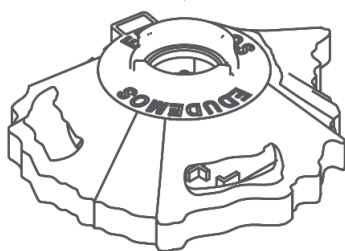
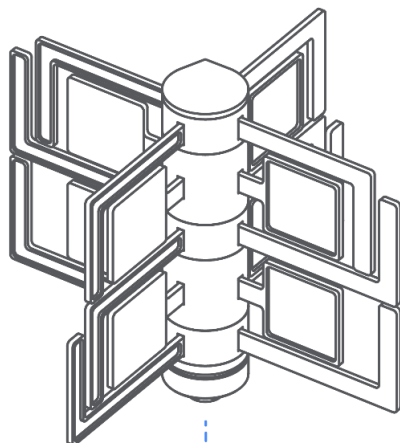




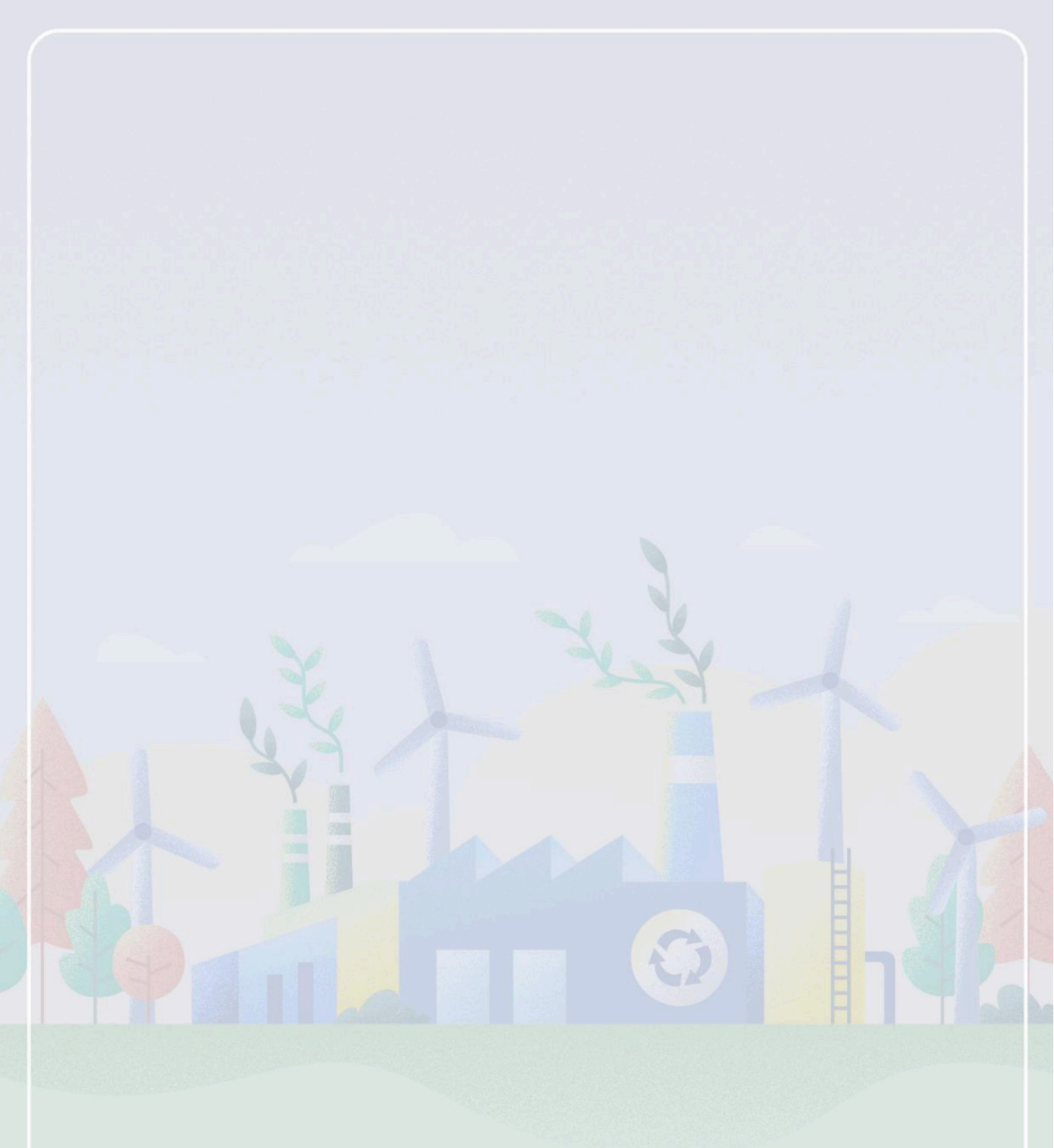




8



Co-funded by
the European Union



EDUDEMOS

EDUcating through Sustainable DEMOnstrators



Cofinanciado por
la Unión Europea

info@edudemos.eu

 [@edudemos](https://twitter.com/@edudemos)

 [@edudemos](https://www.linkedin.com/company/@edudemos)

www.edudemos.eu